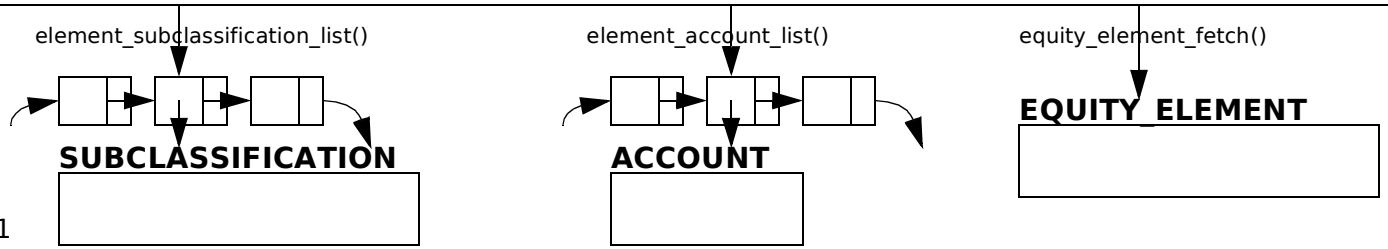
PredictiveBooks Data Structure

ELEMENT

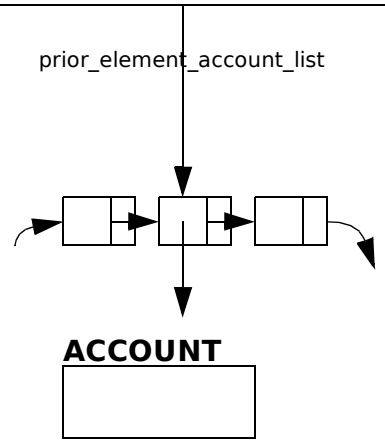
```
/* Usage */
LIST *element_list(
        LIST *filter_element_name_list,
        char *fund_name,
        char *transaction_date_time_nominal,
        char *transaction_date_time_fixed,
        boolean fetch_subclassification_list,
        boolean fetch_account_list );

/* Attributes */
char *element_name;
boolean accumulate_debit;

/* Process */
double element_current_balance;
int percent_of_asset;
int percent_of_revenue;
LIST *subclassification_list;
LIST *account_list;

char *element_filter_where(
        LIST *filter_element_name_list );

char *element_system_string(
        char *element_filter_where() );

LIST *element_system_list(
        char *element_system_string(),
        char *fund_name,
        char *transaction_date_time_nominal,
        char *transaction_date_time_fixed,
        boolean fetch_subclassification_list,
        boolean fetch_account_list );

ELEMENT *element_parse(
        char *input,
        char *fund_name,
        char *transaction_date_time_nominal,
        char *transaction_date_time_fixed,
        boolean fetch_subclassification_list,
        boolean fetch_account_list );

LIST *element_subclassification_list(
        double *element_current_balance,
        char *element_name,
        char *fund_name,
        char *transaction_date_time );

LIST *element_account_list(
        double *element_current_balance,
        char *element_name,
        char *fund_name,
        char *transaction_date_time );

ELEMENT *element_seek(
        char *element_name,
        LIST *element_list() );

ACCOUNT *element_account_seek(
        char *account_name,
        LIST *element_list() );

SUBCLASSIFICATION *element_subclassification_seek(
        char *subclassification_name,
        LIST *element_list() );

EQUITY_ELEMENT *equity_element =
        equity_element_fetch(
            ELEMENT *element_seek(
                ELEMENT_EQUITY,
                element_list() )
                    ->element_name,
                    ->element_current_balance,
            char *fund_name,
            char *transaction_date_time );

void element_list_account_action_string_set(
        LIST *element_list,
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        char *begin_date_string,
        char *as_of_date );

void element_list_total_set(
        LIST *element_list );

void element_list_percent_of_asset_set(
        LIST *element_list );

void element_list_percent_of_revenue_set(
        LIST *element_list );

void element_denominator_percent_of_asset_set(
        LIST *element_list,
        double asset_total );

void element_denominator_percent_of_revenue_set(
        LIST *element_list,
        double revenue_total );

void element_list_delta_prior_set(
        LIST *prior_year_element_list,
        LIST *preclose_element_list );

void element_prior_year_element_list_delta_prior_set(
        LIST *prior_year_element_list,
        ELEMENT *preclose_element );

void element_list_current_balance(
        LIST *element_list,
        double equity_net_income );

double element_current_balance(
        LIST *subclassification_list,
        LIST *account_list );

double element_debit_total(
        LIST *element_list );

double element_credit_total(
        LIST *element_list );

int element_percent_of_total(
        double total,
        double denominator );
```



PredictiveBooks Data Structure

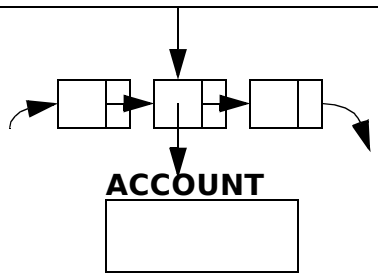
EQUITY_ELEMENT

```
/* Usage */
EQUITY_ELEMENT *equity_element_fetch(
        char *equity_element_name,
        double equity_element_current_balance,
        char *fund_name,
        char *begin_date_string );

/* Input */
char *element_name;
double current_balance;
char *fund_name;
char *begin_date_string;

/* Process */
char *prior_transaction_date_time =
        date_display19(
                date_decrement_second(
                        date_yyyy_mm_dd_new(
                                begin_date_string ),
                        1 ) );

double prior_balance;

LIST *prior_element_account_list =
        element_account_list(
                &prior_balance,
                element_name,
                fund_name,
                prior_transaction_date_time );

double equity_element_balance_change(
        double prior_balance,
        double current_balance );
```



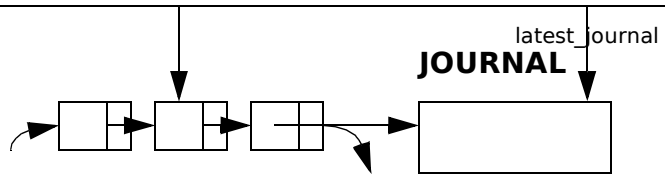
SUBCLASSIFICATION

```
/* Usage */
SUBCLASSIFICATION *subclassification_total_fetch(
        double *subclassification_balance,
        char *subclassification_name,
        char *fund_name,
        char *transaction_date_time_closing() );

/* Attributes */
char *subclassification_name;

/* Process */
double subclassification_balance;
int percent_of_asset;
int percent_of_revenue;
boolean display_if_zero;
char *alternate_display_label;

LIST *account_list =
        subclassification_total_account_list(
            &subclassification_balance,
            subclassification_name,
            fund_name,
            transaction_date_time_closing() );

void subclassification_list_percent_of_asset_set(
        LIST *subclassification_list,
        double asset_total );

void subclassification_list_percent_of_revenue_set(
        LIST *subclassification_list,
        double revenue_total );

void subclassification_list_delta_prior_set(
        LIST *prior_subclassification_list,
        LIST *subclassification_list );

void subclassification_delta_prior_set(
        LIST *prior_subclassification_list,
        SUBCLASSIFICATION *subclassification );

int delta_prior =
        statement_delta_prior(
            prior_subclassification->subclassification_balance,
            subclassification->subclassification_balance );

double subclassification_balance =
        subclassification_list_balance(
            subclassification_list );

double subclassification_debit_total(
        LIST *subclassification_list );

double subclassification_credit_total(
        LIST *subclassification_list );

LIST *subclassification_account_list(
        LIST *subclassification_list );

boolean subclassification_list_populated(
        LIST *subclassification_list );

char *subclassification_label(
        char *subclassification_name,
        char *alternate_display_label,
        boolean with_total );
```



ACCOUNT

```
/* Attributes */
char *account_name;
char *fund_name;
char *subclassification_name;
char *hard_coded_account_key;
boolean accumulate_debit;
double balance;
double payment_amount;

/* Process */
LIST *journal_list;
JOURNAL *latest_journal;
double account_balance;
int percent_of_asset;
int percent_of_revenue;
boolean display_if_zero;

char *account_action_string(
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        char *begin_date_string,
        char *as_of_date,
        char *account_name );

void account_list_percent_of_asset_set(
        LIST *account_list,
        double asset_total );

void account_list_percent_of_revenue_set(
        LIST *account_list,
        double revenue_total );

void account_list_delta_prior_set(
        LIST *prior_account_list,
        LIST *account_list );

void account_delta_prior_set(
        LIST *prior_account_list,
        ACCOUNT *account );

int delta_prior =
        statement_delta_prior(
            prior_account->
                latest_journal->
                balance,
            account->
                latest_journal->
                balance );

double account_balance =
        account_list_balance(
            LIST *account_list );

double account_debit_total(
        LIST *account_list );

double account_credit_total(
        LIST *account_list );

char *account_element_name(
        subclassification_name );
```

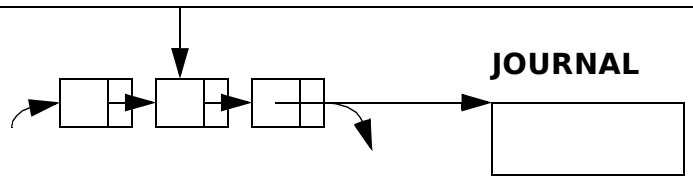


JOURNAL

```
char *full_name;
char *street_address;
char *transaction_date_time;
char *account_name;
double previous_balance;
double debit_amount;
double credit_amount;
double balance;
int transaction_count;
boolean match_sum_taken;
char *transaction_memo;

JOURNAL *journal_new(
        full_name,
        street_address,
        transaction_date_time,
        account_name );

char *journal_select(
        void );

JOURNAL *journal_parse(
        input );

LIST *journal_list(
        full_name,
        street_address,
        transaction_date_time );

LIST *journal_list_fetch(
        where_clause );

JOURNAL *journal_prior(
        transaction_date_time,
        account_name );
FILE *journal_insert_open(
        void );

void journal_insert(
        insert_pipe,
        full_name,
        street_address,
        transaction_date_time,
        account_name,
        amount,
        is_debit );

/* Also does journal_propagate() for each account */
void journal_delete(
        full_name,
        street_address,
        transaction_date_time );

void journal_propagate(
        transaction_date_time,
        account_name );

LIST *journal_list_minimum(
        minimum_transaction_date_time,
        account_name );

LIST *journal_list_account(
        account_name );

void journal_list_set_balances(
        journal_list_prior(),
        account_accumulate_debit(
            account_name ) );

void journal_list_propagate_update(
        journal_list_prior() );

JOURNAL *journal_account_fetch(
        account_name,
        transaction_date_time );

LIST *journal_account_name_list(
        full_name,
        street_address,
        transaction_date_time );

LIST *journal_binary_list(
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        debit_account,
        credit_account );

LIST *journal_year_list(
        int year,
        char *account_name,
        boolean fetch_memo );
```

TRANSACTION (1)

```
char *full_name;
char *street_address;
char *transaction_date_time;
double transaction_amount;
char *memo;
int check_number;
boolean lock_transaction;
char *property_street_address;
char *program_name;
LIST *journal_list;

TRANSACTION *transaction_calloc(
        void );

TRANSACTION *transaction_new(
        full_name,
        street_address,
        transaction_date_time );

TRANSACTION *transaction_fetch(
        full_name,
        street_address,
        transaction_date_time );

char *transaction_primary_where(
        full_name,
        street_address,
        transaction_date_time );

void transaction_refresh(
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        memo,
        check_number,
        lock_transaction,
        journal_list );

/* Returns inserted transaction_date_time */
char *transaction_insert(
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        memo,
        check_number,
        lock_transaction );

FILE *transaction_insert_pipe_open(
        void );

/* Returns inserted transaction_date_time */
char *transaction_insert_pipe(
        insert_pipe,
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        memo,
        check_number,
        lock_transaction,
        journal_list );

/* Returns transaction_list */
LIST *transaction_list_insert(
        transaction_list );

void transaction_list_journal_insert(
        transaction_list );

void transaction_update(
        transaction_amount,
        full_name,
        street_address,
        transaction_date_time );

char *transaction_select(
        void );

char *transaction_primary_where(
        full_name,
        street_address,
        transaction_date_time );

TRANSACTION *transaction_parse(
        input_buffer );

/* Performs journal_propagate() */
void transaction_delete(
        full_name,
        street_address,
        transaction_date_time );

TRANSACTION *transaction_seek(
        transaction_list,
        full_name,
        street_address,
        transaction_date_time );

/* Returns race-free transaction_date_time */
char *transaction_race_free(
        transaction_date_time );
```

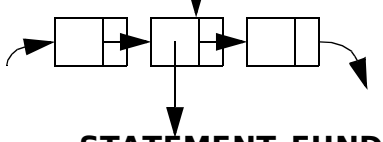


TRANSACTION (2)

```
TRANSACTION *transaction_binary(
        full_name,
        street_address,
        transaction_date_time,
        double transaction_amount,
        char *memo,
        char *debit_account,
        char *credit_account );

char *transaction_memo(
        memo );

void transaction_check_insert(
        insert_pipe,
        check_number );

void transation_lock_insert(
        insert_pipe,
        lock_transaction );

boolean transaction_exists(
        transaction_date_time );

char *transaction_display(
        transaction );

char *transaction_escape_full_name(
        full_name );

TRANSACTION *transaction_check_seek(
        transaction_list,
        check_number );

boolean transaction_closing_entry_exists(
        char *transaction_date );

char *transaction_date_time_closing(
        char *transaction_date,
        boolean preclose_time,
        boolean transaction_closing_entry_exists() );
```

STATEMENT (1)

```
/* Usage */
STATEMENT *statement_new(
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        char *process_name,
        boolean exists_logo_filename,
        LIST *filter_element_name_list,
        char *as_of_date,
        int prior_year_count,
        LIST *fund_name_list,
        char *subclassification_option_string,
        char *fund_aggregation_string,
        char *output_medium_string );

STATEMENT *statement_steady_state(
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        char *process_name,
        boolean exists_logo_filename,
        LIST *filter_element_name_list,
        char *as_of_date,
        int prior_year_count,
        LIST *fund_name_list,
        char *subclassification_option_string,
        char *fund_aggregation_string,
        char *output_medium_string,
        boolean with_postclose,
        STATEMENT *statement );

enum subclassification_option
        statement_subclassification_option(
            subclassification_option_string );

enum fund_aggregation
        statement_fund_aggregation(
            fund_name_list,
            fund_aggregation_string );

enum output_medium
        statement_output_medium(
            output_medium_string );

LIST *statement_fund_name_list( void );

char *statement_begin_date_string(
        char *as_of_date );

char *statement_title(
        char *application_name,
        boolean exists_logo_filename,
        char *process_name );

char *statement_subtitle(
        statement_begin_date_string(),
        char *as_of_date,
        enum fund_aggregation );

LIST *statement_fund_list(
        application_name,
        session,
        login_name,
        role_name,
        filter_element_name_list,
        statement_begin_date_string(),
        as_of_date,
        prior_year_count,
        fund_name_list,
        char *progam_name,
        statement_subclassification_option(),
        statement_fund_aggregation(),
        boolean with_postclose );

boolean statement_exists_closing_entry(
        char *as_of_date );

void statement_account_delta_list(
        char *account_name,
        LIST *prior_year_list );

void statement_html_account_delta_list(
        char *account_name,
        LIST *prior_year_list );

void statement_subclassification_delta_list(
        char *subclassification_name,
        LIST *prior_year_list );

void statement_html_subclassification_delta_list(
        char *subclassification_name,
        LIST *prior_year_list );

void statement_PDF_account_delta_list(
        char *account_name,
        LIST *prior_year_list );

void statement_PDF_subclassification_delta_list(
        char *subclassification_name,
        LIST *prior_year_list );

int statement_delta_prior(
        double prior_total,
        double total );
```



STATEMENT_FUND

statement_subclassification_option

subclassification_aggregate,
subclassification_display,
subclassification_omit

statement_fund_aggregation

single_fund,
sequential,
consolidated,
no_fund

statement_output_medium

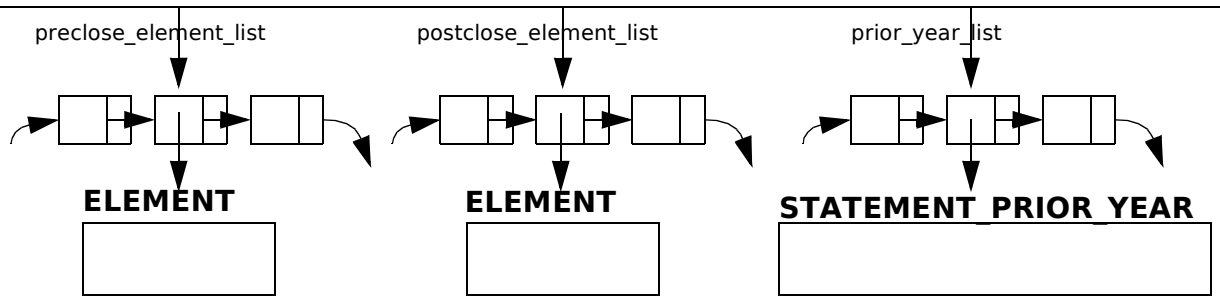
output_table,
output_PDF
output_spreadsheet,
output_stdout

STATEMENT (2)

```
/* Process */
SUBCLASSIFICATION *statement_net_income_subclassification(
        double *equity_element_current_balance,
        int *equity_element_percent_of_asset,
        SUBCLASSIFICATION *last_subclassification,
        EQUITY_ELEMENT *equity_element,
        char *fund_name,
        char *transaction_date_time_closing,
        double asset_element_current_balance,
        boolean is_financial_position );

ELEMENT *statement_equity_liability_element(
        double equity_element_current_balance,
        double asset_element_current_balance,
        double liability_element_current_balance );
```

STATEMENT_FUND (1)

```
/* Usage */
STATEMENT_FUND *statement_fund_fetch(
    char *application_name,
    char *session,
    char *login_name,
    char *role_name,
    LIST *filter_element_name_list,
    char *begin_date_string,
    char *as_of_date,
    int prior_year_count,
    char *fund_name,
    enum subclassification_option,
    boolean with_postclose );

/* Process */
char *transaction_date_time_nominal =
    transaction_date_time_closing(
        as_of_date,
        1 /* preclose_time_boolean */,
        transaction_closing_entry_exists(
            as_of_date ) );

char *transaction_date_time_fixed =
    transaction_date_time_closing(
        as_of_date,
        0 /* not preclose_time_boolean */,
        transaction_closing_entry_exists(
            as_of_date ) );

LIST *statement_fund_element_list(
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        LIST *filter_element_name_list,
        char *begin_date_string,
        char *transaction_date_time_nominal,
        char *transaction_date_time_fixed,
        char *fund_name,
        enum subclassification_option );

LIST *preclose_element_list =
        statement_fund_element_list(
            application_name,
            session,
            login_name,
            role_name,
            filter_element_name_list,
            begin_date_string,
            transaction_date_time_nominal,
            transaction_date_time_nominal,
            fund_name,
            subclassification_option );

LIST *postclose_element_list =
        statement_fund_element_list(
            application_name,
            session,
            login_name,
            role_name,
            filter_element_name_list,
            begin_date_string,
            transaction_date_time_fixed,
            transaction_date_time_fixed,
            fund_name,
            subclassification_option );

LIST *prior_year_list;

for years_ago = 1 to prior_year_count
{
        list_set(
            prior_year_list,
            statement_prior_year_new(
                application_name,
                session,
                login_name,
                role_name,
                filter_element_name_list,
                as_of_date,
                years_ago,
                fund_name,
                subclassification_option ) );
}

void statement_fund_list_steady_state(
        LIST *statement_fund_list,
        char *title,
        char *subtitle,
        boolean exists_logo_filename );

STATEMENT_FUND *statement_fund_steady_state(
        LIST *preclose_element_list,
        LIST *postclose_element_list,
        char *title,
        char *subtitle,
        boolean exists_logo_filename,
        STATEMENT_FUND *statement_fund );

void statement_fund_list_prior_year_list_set(
        LIST *statement_fund_list,
        int prior_year_count );

LIST *statement_fund_steady_state_prior_year_list(
        int prior_year_count,
        LIST *preclose_element_list,
        STATEMENT_FUND *statement_fund );

STATEMENT_FUND *statement_fund_seek(
        char *fund_name,
        LIST *statement_fund_list );
```

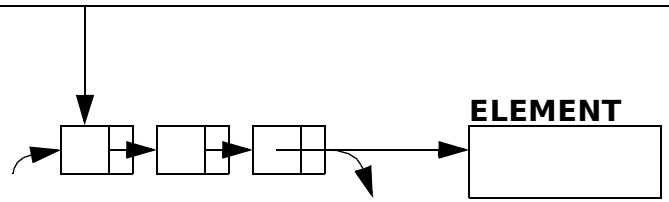


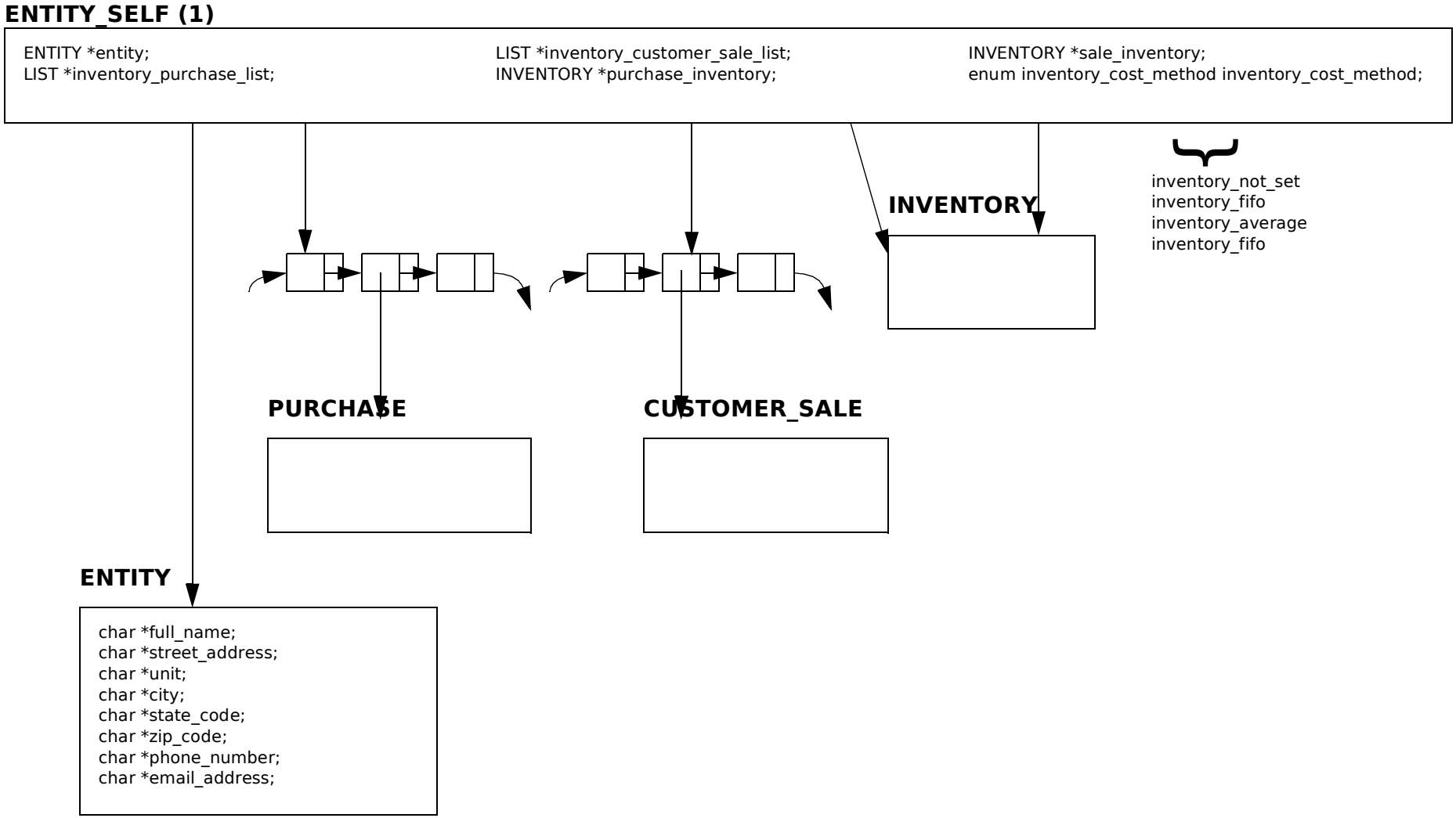
STATEMENT_FUND (2)

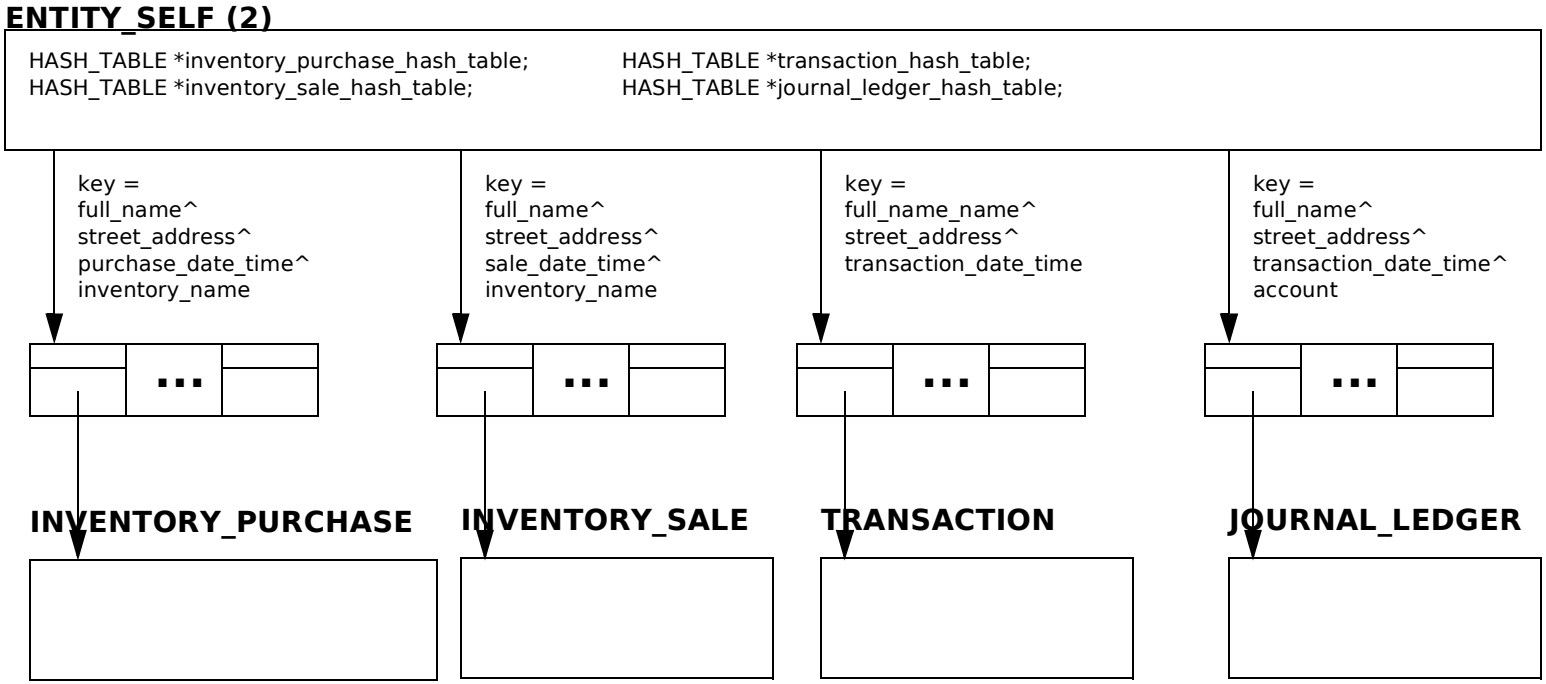
```
/* Process */
double preclose_debit_total =
        element_list_debit_total(
            preclose_element_list );

double preclose_credit_total =
        element_list_credit_total(
            preclose_element_list );

double postclose_debit_total =
        element_list_debit_total(
            postclose_element_list );

double postclose_credit_total =
        element_list_credit_total(
            postclose_element_list );

boolean statement_fund_exists_postclose(
        LIST *statement_fund_list );

void statement_fund_list_net_income_set(
    LIST *statement_fund_list );

void statement_fund_list_net_income_fetch(
    LIST *statement_fund_list );

double statement_fund_net_income_fetch(

boolean  statement_fund_exists_prior_year(
        LIST *statement_fund_list );

double statement_fund_total(
    LIST *subclassification_list,
    LIST *account_list );

double revenue_total =
        element_seek(
            ELEMENT_REVENUE,
            preclose_element_list )->
                element_current_balance() );

double expense_total =
        element_seek(
            ELEMENT_EXPENSE,1
            preclose_element_list )->
                element_current_balance() );

double gain_total =
        element_seek(
            ELEMENT_GAIN,
            preclose_element_list )->
                element_current_balance() );

double loss_total =
        element_seek(
            ELEMENT_LOSS,
            preclose_element_list )->
                element_current_balance() );

double net_income =
        statement_fund_net_income(
            revenue_total,
            expense_total,
            gain_total,
            loss_total );

int statement_fund_net_income_percent(
        statement_fund_net_income(),
        revenue_total );

char *statement_fund_caption(
        char *statement_title(),
        char *statement_subtitle(),
        char *statement_fund->fund_name,
        boolean exists_logo_filename );

void statement_fund_list_equity_set(
        LIST *statement_fund_list,
        boolean is_financial_position,
        enum subclassification_option );

void statement_fund_display_equity(
        double *element_current_balance,
        int *percent_of_asset,
        element->subclassification_list /* out */,
        element->equity_element /* in/out */,
        double net_income,
        double asset_current_balance,
        boolean is_financial_position );

void statement_fund_omit_equity(
        double *element_current_balance,
        int *percent_of_asset,
        LIST *account_list /* out */,
        element->equity_element /* in/out */,
        double net_income,
        double asset_current_balance,
        boolean is_financial_position );

void statement_fund_aggregate_equity(
        double *element_current_balance,
        int *percent_of_asset,
        LIST *subclassification_list /* out */,
        element->equity_element /* in/out */,
        double net_income,
        double asset_current_balance,
        boolean is_financial_position );
```

STATEMENT_PRIOR_YEAR

```
/* Usage */
STATEMENT_PRIOR_YEAR *
    statement_prior_year_new(
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        LIST *filter_element_name_list,
        char *as_of_date,
        int years_ago,
        char *fund_name,
        enum subclassification_option );

/* Attributes */
char *application_name;
char *session;
char *login_name;
char *role_name;
LIST *filter_element_name_list;
char *as_of_date;
int years_ago;
char *fund_name;
enum subclassification_option subclassification_option;

/* Process */
char *prior_date_string =
    strdup(
        date_yyyy_mm_dd_display(
            date_subtract_year(
                date_yyyy_mm_dd_new(
                    as_of_date ),
                years_ago ) ) );

char *transaction_date_time_nominal =
    transaction_date_time_closing(
        prior_date_string,
        1 /* preclose_time_boolean */,
        transaction_closing_entry_exists(
            prior_date_string ) );

char *transaction_date_time_fixed =
    transaction_date_time_closing(
        prior_date_string,
        0 /* not preclose_time_boolean */,
        transaction_closing_entry_exists(
            prior_date_string ) );

char *begin_date_string =
    statement_begin_date_string(
        prior_date_string );

LIST *prior_year_element_list =
    statement_fund_element_list(
        application_name,
        session,
        login_name,
        role_name,
        filter_element_name_list,
        begin_date_string,
        transaction_date_time_nominal,
        transaction_date_time_fixed,
        fund_name,
        subclassification_option );

LIST *statement_prior_year_heading_list(
    LIST *prior_year_list );

LIST *statement_prior_year_delta_list(
    char *account_name,
    LIST *prior_year_list );

void statement_prior_year_list_net_income_set(
        LIST *prior_year_list,
        statement_fund_net_income() );

void statement_prior_year_net_income_set(
        STATEMENT_PRIOR_YEAR
            *statement_prior_year,
        statement_fund_net_income );

double revenue_total =
        element_seek(
            ELEMENT_REVENUE,
            prior_year_element_list )->
                element_current_balance() );

double expense_total =
        element_seek(
            ELEMENT_EXPENSE,
            prior_year_element_list )->
                element_current_balance() );

double gain_total =
        element_seek(
            ELEMENT_GAIN,
            prior_year_element_list )->
                element_current_balance() );

double loss_total =
        element_seek(
            ELEMENT_LOSS,
            prior_year_element_list )->
                element_current_balance() );

double statement_prior_year_net_income =
        statement_fund_net_income(
            revenue_total,
            expense_total,
            gain_total,
            loss_total );

int delta_prior =
        statement_delta_prior(
            statement_prior_year_net_income,
            statement_fund_net_income );
```

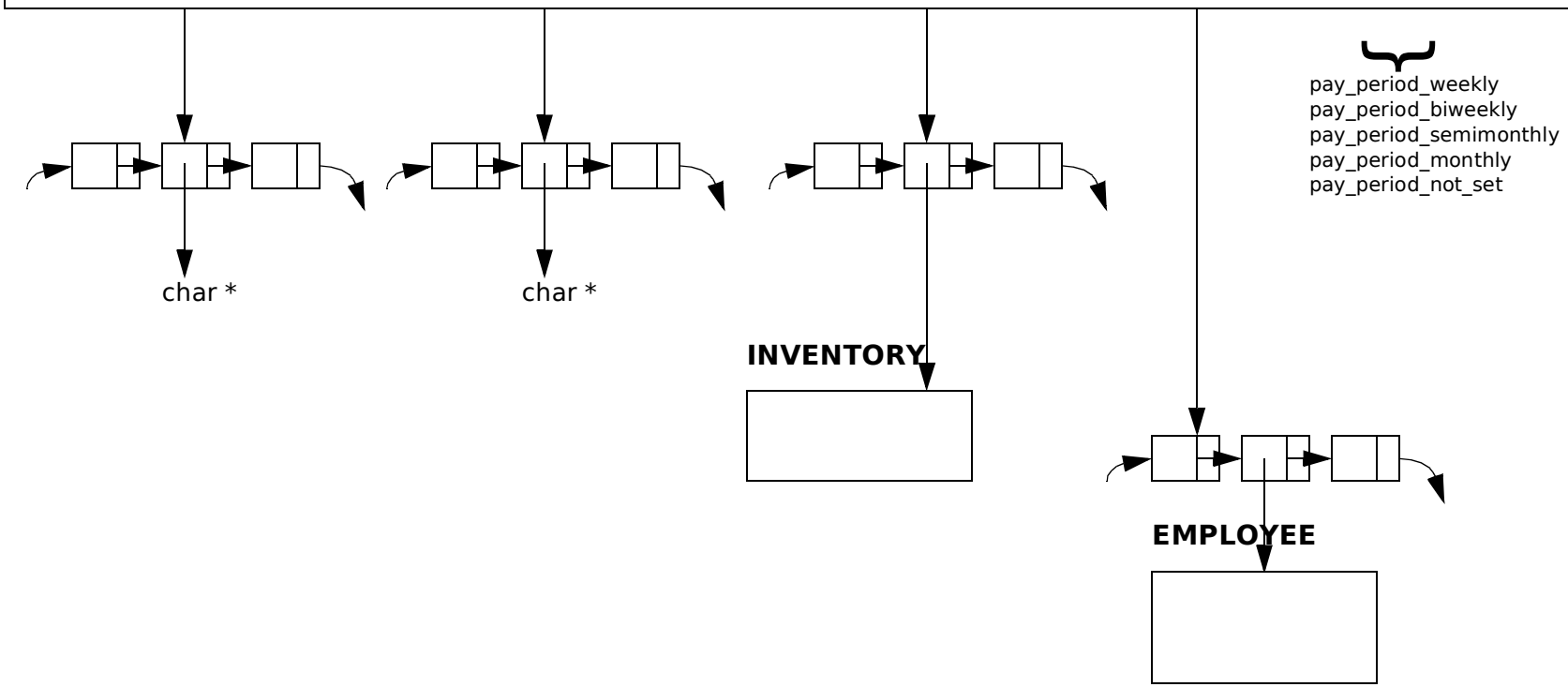



ENTITY_SELF (1)
ENTITY *entity;
LIST *inventory_purchase_list;
LIST *inventory_customer_sale_list;
INVENTORY *purchase_inventory;
INVENTORY *sale_inventory;
enum inventory_cost_method inventory_cost_method;
inventory_not_set
inventory_fifo
inventory_average
inventory_fifo
INVENTORY
PURCHASE
CUSTOMER_SALE
ENTITY
char *full_name;
char *street_address;
char *unit;
char *city;
char *state_code;
char *zip_code;
char *phone_number;
char *email_address;


ENTITY_SELF (2)
HASH_TABLE *inventory_purchase_hash_table;
HASH_TABLE *inventory_sale_hash_table;
HASH_TABLE *transaction_hash_table;
HASH_TABLE *journal_ledger_hash_table;
key =
full_name^
street_address^
purchase_date_time^
inventory_name
key =
full_name^
street_address^
sale_date_time^
inventory_name
key =
full_name_name^
street_address^
transaction_date_time
key =
full_name^
street_address^
transaction_date_time^
account
...
...
...
...
INVENTORY_PURCHASE
INVENTORY_SALE
TRANSACTION
JOURNAL_LEDGER

ENTITY_SELF (3)

```
LIST *inventory_purchase_name_list;
LIST *inventory_sale_name_list;
LIST *inventory_list;
LIST *employee_list;
enum payroll_pay_period payroll_pay_period;
```



ENTITY_SELF (4)

```
char *payroll_beginning_day
double social_security_combined_tax_rate
int social_security_payroll_ceiling
double medicare_combined_tax_rate
double medicare_additional_withholding_rate
int medicare_additional_gross_pay_floor
double federal_withholding_allowance_period_value
double federal_nonresident_withholding_income_premium
double state_withholding_allowance_period_value
double state_itemized_allowance_period_value
int federal_unemployment_wage_base
double federal_unemployment_tax_standard_rate
double federal_unemployment_threshold_rate
double federal_unemployment_tax_minimum_rate
int state_unemployment_wage_base
double state_unemployment_tax_rate
double state_sales_tax_rate;

char *entity_self_paypal_cash_account_name();
```

SALE (1)

```
ENTITY *customer_entity =
        entity_fetch(
            full_name,
            street_address );

char *sale_date_time;
char *completed_date_time;
double shipping_revenue;
enum title_passage_rule title_passage_rule;

/* Returns sale_steady_state() */
SALE *sale_fetch(
        full_name,
        street_address,
        sale_date_time );

char *sale_primary_where(
        full_name,
        street_address,
        sale_date_time );

double sale_gross_revenue(
        inventory_sale_total(),
        fixed_service_sale_total(),
        hourly_service_sale_total() );

double sale_sales_tax(
        inventory_sale_list(),
        entity_self_sales_tax_rate() );

double sale_invoice_amount(
        sale_gross_revenue(),
        sale_sales_tax(),
        shipping_charge );

double Sale_amount_due(
        sale_invoice_amount(),
        customer_payment_total() )
{
        return A - B;
}

TRANSACTION *sale_transaction(
        full_name,
        street_address,
        completed_date_time,
        invoice_amount,
        gross_revenue,
        sale_tax,
        shipping_charge,
        account_receivable(),
        account_revenue(),
        account_shipping_revenue(),
        account_sales_tax_payable() );

void sale_update(
        inventory_sale_total(),
        fixed_service_sale_total(),
        hourly_service_sale_total(),
        sale_gross_revenue()
        sale_sales_tax(),
        sale_invoice_amount(),
        payment_total(),
        sale_amount_due(),
        sale_transaction()->
            transaction_date_time,
        full_name,
        street_address,
        sale_date_time );

SALE *sale_steady_state(
        full_name,
        street_address,
        sale_date_time,
        inventory_sale_list(),
        fixed_service_sale_list(),
        hourly_service_sale_list(),
        customer_payment_list(),
        shipping_charge,
        entity_self_sales_tax_rate(),
        completed_date_time,
        account_receivable(),
        account_revenue(),
        account_shipping_revenue(),
        account_sales_tax_payable() );
```

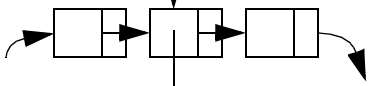
TRANSACTION

ENTITY

FOB_shipping
FOB_destination
title_passage_rule_null

SALE (2)

```
LIST *customer_payment_list(
        full_name,
        street_address,
        sale_date_time );
```

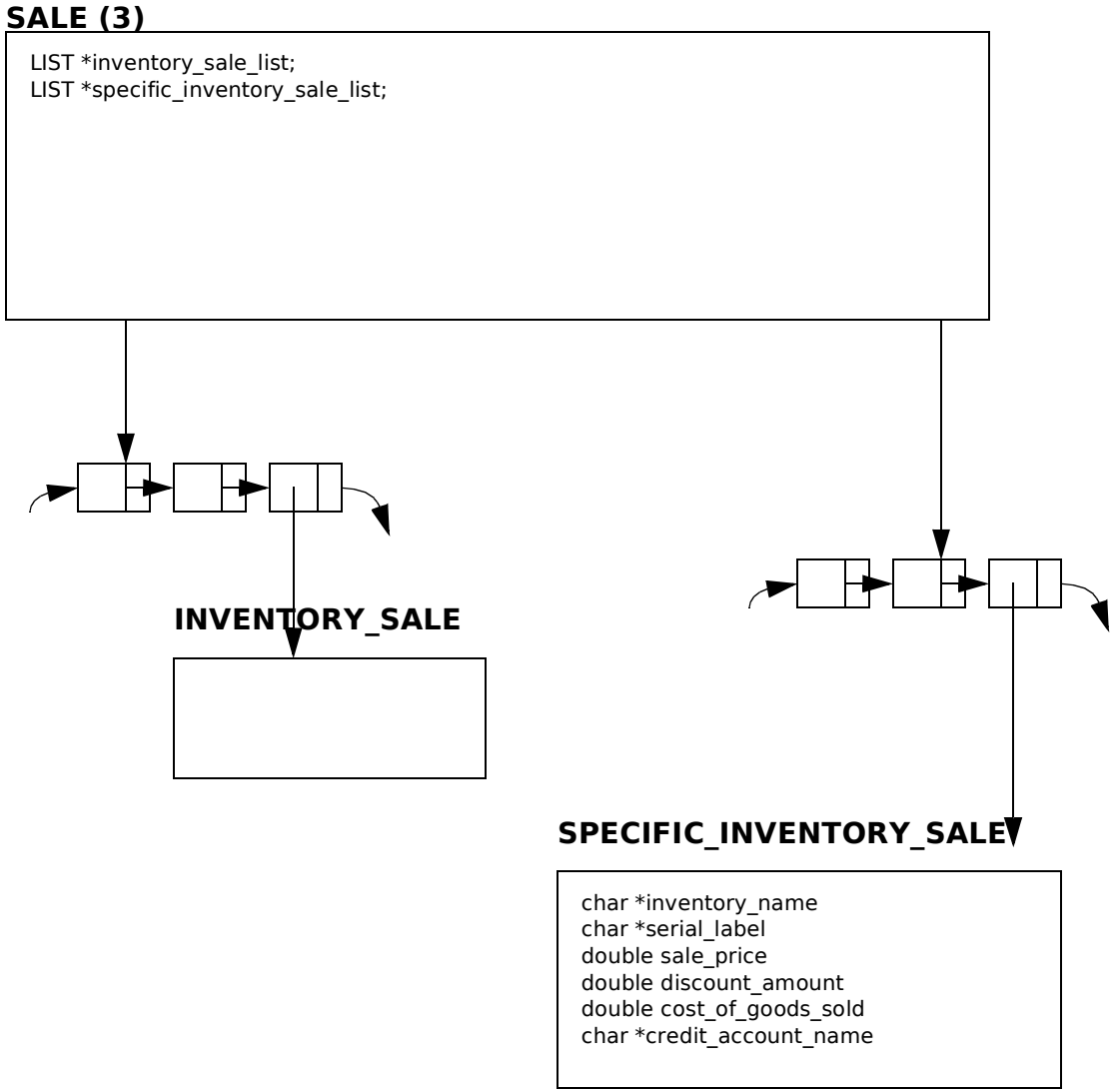


CUSTOMER_PAYMENT

```
ENTITY *customer_entity =
        entity_fetch(
            full_name,
            street_address );

char *sale_date_time;
char *payment_date_time;
double payment_amount;
int check_number;

double customer_payment_total(
        customer_payment_list() )
{
        return sum( A->payment_amount );
}

CUSTOMER_PAYMENT *customer_payment_new(
        full_name,
        street_address,
        sale_date_time,
        payment_date_time );

char *customer_payment_select(
        void );

char *customer_payment_sys_string(
        where );

LIST *customer_payment_system_list(
        sys_string );

CUSTOMER_PAYMENT *customer_payment_parse(
        input );

TRANSACTION *customer_payment_transaction(
        full_name,
        street_address,
        payment_date_time,
        payment_amount,
        account_cash(),
        account_receivable() );
```

ENTITY

TRANSACTION


SALE (3)
LIST *inventory_sale_list;
LIST *specific_inventory_sale_list;
INVENTORY_SALE
SPECIFIC_INVENTORY_SALE
char *inventory_name
char *serial_label
double sale_price
double discount_amount
double cost_of_goods_sold
char *credit_account_name

PredictiveBooks Data Structure

HOURLY_SERVICE_SALE

```
ENTITY *customer_entity =
        entity_new(
            full_name,
            street_address );
char *sale_date_time;
char *service_name;
char *service_description;
double hourly_rate;
double discount_amount;
int estimated_hours;

char *hourly_service_sale_revenue_account_name(
        service_name,
        service_description );

LIST *hourly_service_work_list =
        work_hourly_list(
            full_name,
            street_address,
            sale_date_time,
            service_name,
            service_description );

double hourly_service_sale_work_hours =
        work_list_hours(
            hourly_service_work_list() );

double hourly_service_sale_net_revenue(
        hourly_rate,
        hourly_service_sale_work_hours,
        discount_amount )
{
        return (A x B) - C;
}
```

SALE (4)

```
LIST *    fixed_service_sale_list
LIST *    hourly_service_sale_list
```

FIXED_SERVICE_SALE

```
ENTITY *customer_entity =
        entity_new(
            full_name,
            street_address );
char *sale_date_time;
char *service_name;
double fixed_price;
double discount_amount;
int estimated_hours;
char *fixed_service_sale_revenue_accouint_name(
        service_name );

LIST *fixed_service_sale_work_list =
        work_fixed_list(
            full_name,
            street_address,
            sale_date_time,
            service_name );

double fixed_service_sale_work_hours =
        work_list_hours(
            fixed_service_sale_work_list() );

double fixed_service_sale_net_revenue(
        fixed_price,
        discount_amount )
{
        return A - B;
}
```

WORK

```
ENTITY *customer_entity = entity_new();
char *sale_date_time;
char *service_name;
char *service_description;
char *begin_date_time;
char *end_date_time;
char *login_name;
char *activity_name;

double work_hours(
        begin_date_time,
        end_date_time );

double work_list_hours(
        work_list );
```

PURCHASE (1)

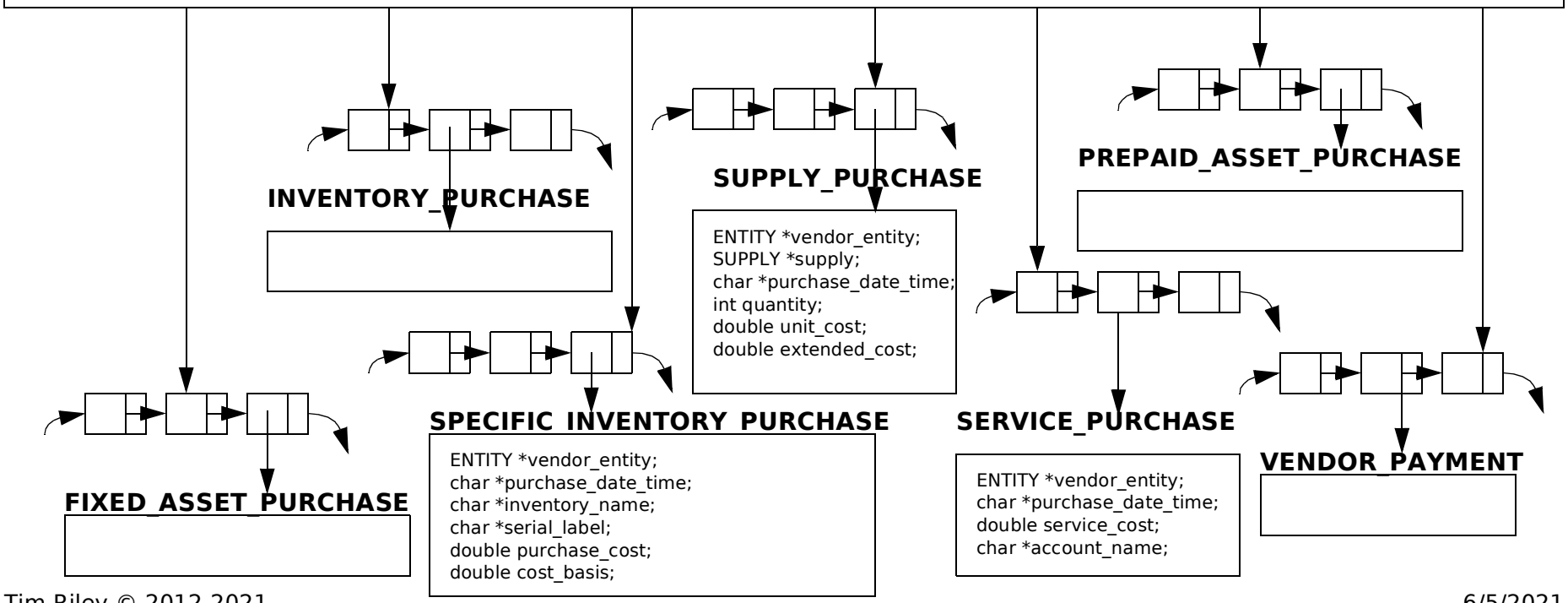
```
/* Usage */
PURCHASE *purchase_fetch(
        char *full_name,
        char *street_address,
        char *purchase_date_time );

/* Input */
ENTITY *vendor_entity =
        entity_new(
            full_name,
            street_address );

char *purchase_date_time;
double sales_tax;
double freight_in;
char *title_passage_rule_string;
char *shipped_date;
char *arrived_date_time;
char *program_name;
char *property_street_address;

/* Process */
enum title_passage_rule title_passage_rule =
        predictive_title_passage_rule_resolve(
            title_passage_rule_string );

char *purchase_primary_where(
        char *full_name,
        char *street_address,
        char *purchase_date_time );

char *purchase_system_string(
        char *where,
        char *order );

LIST *purchase_system_list(
        purchase_system_string() );

PURCHASE *purchase_parse(
        char *input );

LIST *fixed_asset_purchase_list =
        fixed_asset_purchase_list_fetch(
            vendor_entity->full_name,
            vendor_entity->street_address,
            purchase_date_time );

LIST *vendor_payment_list =
        vendor_payment_list_fetch(
            full_name,
            street_address,
            purchase_date_time );

PURCHASE *purchase_steady_state(
        sales_tax,
        freight_in,
        arrived_date,
        fixed_asset_purchase_list,
        inventory_purchase_list,
        specific_inventory_purchase_list,
        supply_purchase_list,
        prepaid_asset_purchase_list,
        inventory_purchase_return_list,
        vendor_payment_list,
        purchase );

double fixed_asset_purchase_total =
        fixed_asset_purchase_total(
            fixed_asset_purchase_list );

double inventory_purchase_total =
        inventory_purchase_total(
            inventory_purchase_list );

double specific_inventory_purchase_total =
        specific_inventory_purchase_total(
            specific_inventory_purchase_list );

double supply_purchase_total =
        supply_purchase_total(
            supply_purchase_list );

double prepaid_asset_purchase_total =
        prepaid_asset_purchase_total(
            prepaid_asset_purchase_list );

double inventory_purchase_return_total =
        inventory_purchase_return_total(
            inventory_purchase_return_list );

double vendor_payment_total =
        vendor_payment_total(
            vendor_payment_list );

double purchase_invoice_amount(
        fixed_asset_purchase_total,
        inventory_purchase_total,
        specific_inventory_purchase_total,
        supply_purchase_total,
        prepaid_asset_purchase_total,
        sales_tax,
        freight_in );

double purchase_amount_due(
            purchase_invoice_amount,
            vendor_payment_total )
{
        return A - B;
}

double purchase_asset_cost_basis(
        asset_cost,
        sales_tax,
        freight_in,
        fixed_asset_purchase_total(),
        inventory_purchase_total(),
        specific_inventory_purchase_total(),
        supply_expense_purchase_total(),
        prepaid_asset_purchase_total() );
```

ENTITY

FOB_shipping
FOB_destination
title_passage_rule_null

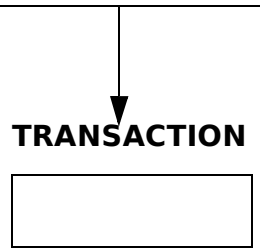
PURCHASE (2)

```
TRANSACTION *purchase_transaction =
        purchase_fixed_asset_transaction(
            full_name,
            street_address,
            arrived_date_time,
            purchase_invoice_amount,
            sales_tax,
            freight_in,
            fixed_asset_purchase_list(),
            account_payable() );

/* Returns true transaction_date_time */
char *purchase_transaction_refresh(
        purchase_transaction->
            transaction_amount,
        purchase_transaction->
            journal_list,
        purchase_transaction->memo,
        full_name,
        street_address,
        purchase_transaction->
            transaction_date_time );

void purchase_update(
        fixed_asset_purchase_total,
        purchase_invoice_amount,
        vendor_payment_total,
        purchase_amount_due,
        purchase_transaction->
            transaction_date_time,
        full_name,
        street_address,
        purchase_date_time );

LIST *  inventory_purchase_list
LIST *  specific_inventory_purchase_list
LIST *  service_purchase_list
LIST *  supply_purchase_list
LIST *  prepaid_asset_purchase_list;
char *property_street_address;
```



VENDOR_PAYMENT

```
/* Usage */
VENDOR_PAYMENT *vendor_payment_fetch(
        char *full_name,
        char *street_address,
        char *purchase_date_time,
        char *payment_date_time );

LIST *vendor_payment_list_fetch(
        full_name,
        street_address,
        purchase_date_time );

/* Input */
ENTITY *vendor_entity;
char *purchase_date_time;
char *payment_date_time;
double payment_amount;
int check_number;

/* Process */
VENDOR_PAYMENT *vendor_payment_new(
        full_name,
        street_address,
        purchase_date_time,
        payment_date_time );

char *vendor_payment_primary_where(
        full_name,
        street_address,
        purchase_date_time,
        payment_date_time );

char *vendor_payment_system_string(
        purchase_primary_where() );

LIST *vendor_payment_system_list(
        vendor_payment_system_string() );

VENDOR_PAYMENT *vendor_payment_parse(
        input );

double vendor_payment_total(
        LIST *vendor_payment_system_list() )
{
        return sum(
                A->
                vendor_payment->
                payment_amount );
}

TRANSACTION *vendor_payment_transaction(
        full_name,
        street_address,
        payment_date_time,
        payment_amount,
        check_number,
        account_payable(),
        account_cash(),
        account_uncleared_checks() );

LIST *vendor_payment_journal_list(
        payment_amount,
        check_number,
        account_payable(),
        account_cash(),
        account_uncleared_checks() );

void vendor_payment_insert(
        full_name,
        street_address,
        purchase_date_time,
        payment_date_time,
        payment_amount,
        check_number );

VENDOR_PAYMENT *vendor_payment_seek(
        vendor_payment_list,
        payment_date_time );

char *vendor_payment_update_system_string(
        void );
```



FIXED_ASSET_PURCHASE

```
/* Usage */
FIXED_ASSET_PURCHASE *fixed_asset_purchase_fetch(
        char *asset_name,
        char *serial_label,
        boolean fetch_last_depreciation,
        boolean fetch_last_recovery );

LIST *fixed_asset_purchase_list_fetch(
        char *where,
        boolean fetch_last_depreciation,
        boolean fetch_last_recovery );

/* Input */
FIXED_ASSET *fixed_asset;
char *serial_label;
ENTITY *vendor_entity;
char *purchase_date_time;
char *service_placement_date;
double fixed_asset_cost;
int units_produced_so_far;
char *disposal_date;
char *cost_recovery_period_string;
char *cost_recovery_method;
char *cost_recovery_conversion;
enum depreciation_method depreciation_method =
        depreciation_method_resolve();
int estimated_useful_life_years;
int estimated_useful_life_units;
int estimated_residual_value;
int declining_balance_n;
double finance_accumulated_depreciation;
double tax_adjusted_basis;

/* Process */
FIXED_ASSET_PURCHASE *fixed_asset_purchase_new(
        char *asset_name,
        char *serial_number );

FIXED_ASSET *fixed_asset =
        fixed_asset_fetch(
            asset_name );

char *fixed_asset_purchase_primary_where(
        char *asset_name,
        char *serial_label );

char *fixed_asset_purchase_system_string(
        char *where,
        char *order );

LIST *fixed_asset_purchase_system_list(
        fixed_asset_purchase_system_string(),
        boolean fetch_last_depreciation,
        boolean fetch_last_recovery );

FIXED_ASSET_PURCHASE *fixed_asset_purchase_parse(
        char *input,
        boolean fetch_last_depreciation,
        boolean fetch_last_recovery );

double cost_basis =
        purchase_cost_basis(
            fixed_asset_cost,
            purchase->sales_tax,
            purchase->freight_in,
            fixed_asset_purchase_total(),
            inventory_purchase_total(),
            specific_inventory_purchase_total(),
            supply_expense_purchase_total(),
            prepaid_asset_purchase_total() );

double finance_accumulated_depreciation =
        depreciation_accumulated_depreciation();

double tax_accumulated_depreciation =
        recovery_accumulated_depreciation();

LIST *fixed_asset_purchase_list_depreciate(
        fixed_asset_purchase_list_fetch(
            "finance_accumulated_depreciation <
             cost_basis",
            0 /* not fetch_last_depreciation */,
            0 /* not fetch_last_recovery */ ),
        date_yyyy_mm_dd_now()
            /* depreciation_date */ );

DEPRECIATION *depreciation =
        depreciation_evaluate(
            fixed_asset->asset_name,
            fixed_asset_purchase->serial_label,
            service_placement_date,
            depreciation_prior_depreciation_date(),
            date_yyyy_mm_dd_now()
                /* depreciation_date */,
            fixed_asset_purchase_cost_basis(),
            units_produced_so_far,
            estimated_residual_value,
            estimated_useful_life_years,
            estimated_useful_life_units,
            declining_balance_n,
            finance_accumulated_depreciation
                /* prior_accumulated_depreciation */,
            enum depreciation_method ) );

LIST *fixed_asset_purchase_depreciation_list(
        LIST *fixed_asset_purchase_list );

void fixed_asset_purchase_list_update(
        LIST *fixed_asset_purchase_list );

FILE *fixed_asset_purchase_update_open(
        void );

void fixed_asset_purchase_update(
        update_pipe,
        fixed_asset_purchase_cost_basis(),
        finance_accumulated_depreciation,
        tax_adjusted_basis,
        asset_name,
        serial_label );

void fixed_asset_purchase_finance_fetch_update(
        char *asset_name,
        char *serial_label );

RECOVERY *recovery =
        recovery_evaluate();

void fixed_asset_purchase_cost_fetch_update(
        char *asset_name,
        char *serial_label );

LIST *fixed_asset_purchase_list_cost_recover(
        LIST *fixed_asset_purchase_list,
        int tax_year );

LIST *fixed_asset_purchase_cost_recovery_list(
        LIST *fixed_asset_purchase_list );
```

ENTITY **FIXED_ASSET** **DEPRECIATION** **RECOVERY**

FIXED_ASSET

```
/* Usage */
FIXED_ASSET *fixed_asset_fetch(
        char *asset_name );

/* Attributes */
char *asset_name;
char *account_name;
char *cost_recovery_period_string;
char *cost_recovery_method;
char *cost_recovery_conversion;
double activity_energy_kilowatt_draw;
double activity_depreciation_per_hour;

/* Process */
FIXED_ASSET *fixed_asset_new(
        char *asset_name );

FIXED_ASSET *fixed_asset_parse(
        char *input );

char *fixed_asset_primary_where(
        char *asset_name );

char *fixed_asset_system_string(
        char *where );

char *fixed_asset_name_escape(
        char *fixed_asset );
```

RECOVERY

```
/* Usage */
RECOVERY *recovery_evaluate(
        char *asset_name,
        char *serial_label,
        int tax_year,
        double fixed_asset_purchase->
            cost_basis,
        char *fixed_asset_purchase->
            service_placement_date,
        char *fixed_asset_purchase->
            disposal_date,
        char *fixed_asset_purchase->
            cost_recovery_period_string,
        char *fixed_asset_purchase->
            cost_recovery_method,
        char *fixed_asset_purchase->
            cost_recovery_conversion );

RECOVERY *recovery_fetch(
        char *asset_name,
        char *serial_label,
        int tax_year );

LIST *recovery_list_fetch(
        char *asset_name,
        char *serial_label );

void recovery_fixed_asset_list_set(
        LIST *fixed_asset_list,
        int tax_year );

/* Input */
char *asset_name;
char *serial_label;
int tax_year;
double cost_basis;
char *service_placement_date;
char *disposal_date;
char *cost_recovery_period_string;
char *cost_recovery_method;
char *cost_recovery_conversion;

/* Process */
char *recovery_system_string(
        char *where );

LIST *recovery_system_list(
        recovery_system_string() );

char *recovery_primary_where(
        char *asset_name,
        char *serial_label,
        int tax_year );

double recovery_period_years(
        char *cost_recovery_period_string );

int service_placement_month =
        date_month(
            date_yyyy_mm_dd_new(
                service_placement_date ) );

int service_placement_year =
        date_year(
            date_yyyy_mm_dd_new(
                service_placement_date ) );

int disposal_month =
        date_month(
            date_yyyy_mm_dd_new(
                disposal_date ) );

int disposal_year =
        date_year(
            date_yyyy_mm_dd_new(
                disposal_date ) );

int recovery_service_placement_years(
        int tax_year,
        int service_placement_year )
{
        return A - B;
}

char *recovery_subquery_where(
        int tax_year );

RECOVERY_STRAIGHT_LINE *straight_line =
        recovery_statutory_straight_line_evaluate(
            tax_year,
            cost_basis,
            service_placement_month,
            service_placement_year,
            disposal_month,
            disposal_year,
            recovery_period_years(),
            cost_recovery_conversion );

RECOVERY_ACCELERATED *accelerated =
        recovery_statutory_accelerated_evaluate(
            tax_year,
            cost_basis,
            service_placement_month,
            service_placement_year,
            disposal_month,
            disposal_year,
            recovery_period_years(),
            cost_recovery_conversion );

void recovery_fixed_asset_list_display(
        FILE *output_pipe,
        LIST *fixed_asset_list );

void recovery_list_insert(
        LIST *recovery_list );

double recovery_rate(
        straight_line,
        accelerated );

double recovery_amount(
        straight_line,
        accelerated );
```

RECOVERY_STRAIGHT_LINE **RECOVERY_ACCELERATED**

RECOVERY_STRAIGHT_LINE

```
/* Usage */
RECOVERY_STRAIGHT_LINE *
        recovery_statutory_straight_line_evaluate(
            int tax_year,
            double cost_basis,
            int service_placement_month,
            int service_placement_year,
            int disposal_month,
            int disposal_year,
            double recovery_period_years,
            char *cost_recovery_conversion );

/* Process */
RECOVERY_STRAIGHT_LINE *recovery_straight_line_calloc();

double service_placement_years =
        recovery_service_placement_years(
            tax_year,
            service_placement_year );

double straight_line_rate =
        recovery_statutory_straight_line_half_year_rate(
            int recovery_service_placement_years(),
            int tax_year,
            int disposal_year,
            double recovery_period_years );

double recovery_statutory_straight_line_half_year_mid_year_rate(
        double recovery_period_years );

double recovery_statutory_straight_line_half_year_book_ends_rate(
        double recovery_statutory_straight_line_mid_year_rate() );

double straight_line_amount =
        recovery_straight_line_amount(
            double straight_line_rate,
            double cost_basis )
{
        return A X B;
}
```

RECOVERY_ACCELERATED

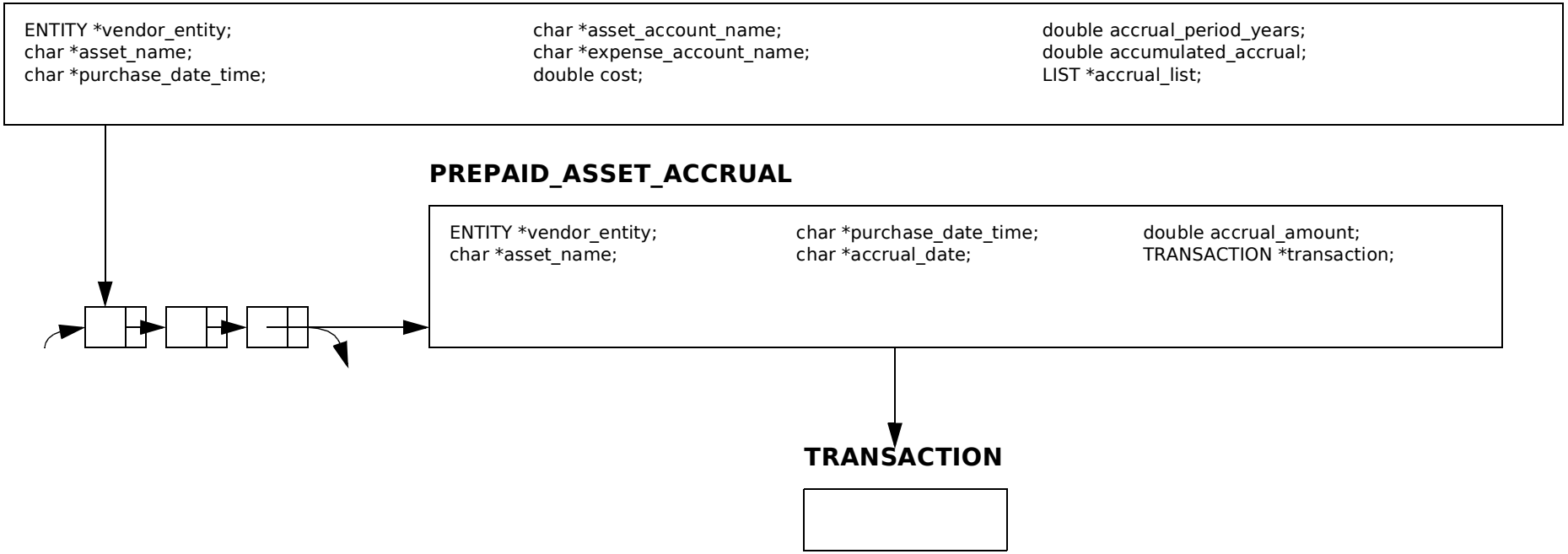
```
/* Usage */
RECOVERY_ACCELERATED *
        recovery_statutory_accelerated_evaluate(
            int tax_year,
            double cost_basis,
            int service_placement_month,
            int service_placement_year,
            int disposal_month,
            int disposal_year,
            double recovery_period_years,
            char *cost_recovery_conversion );

/* Process */
RECOVERY_ACCELERATED *recovery_accelerated_calloc();

double accelerated_rate =
        recovery_statutory_accelerated_half_year_rate(
            int tax_year,
            int disposal_year,
            double recovery_period_years );

double recovery_statutory_accelerated_half_year_rate_evaluate(
        int tax_year,
        double recovery_period_years );

double accelerated_amount =
        recovery_accelerated_amount(
            double accelerated_rate,
            double cost_basis );
```

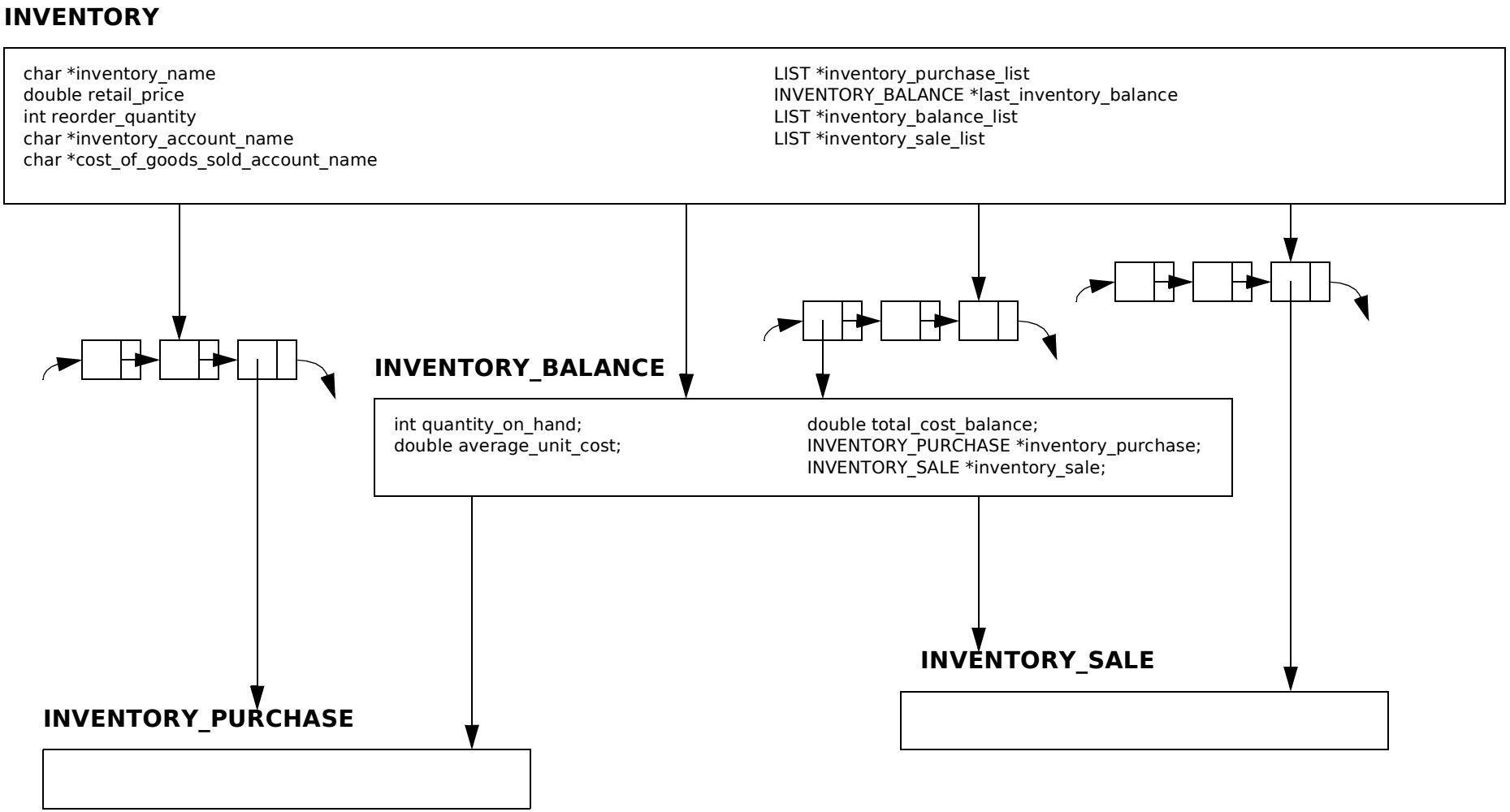
DEPRECIATION

```
/* Usage */
DEPRECIATION *depreciation_evaluate(
        char *asset_name,
        char *serial_label,
        depreciation_method_evaluate(),
        char *service_placement_date,
        char *depreciation_prior_depreciation_date(),
        char *date_yyyy_mm_dd_now()
            /* depreciation_date */,
        double cost_basis,
        int units_produced_so_far,
        int estimated_residual_value,
        int estimated_useful_life_years,
        int estimated_useful_life_units,
        int declining_balance_n,
        double fixed_asset_purchase->
            finance_accumulated_depreciation
              /* prior_accumulated_depreciation */ );

DEPRECIATION *depreciation_fetch(
        char *asset_name,
        char *serial_label,
        char *depreciation_date );

/* Process */
enum depreciation_method =
        depreciation_method_evaluate(
            depreciation_method_string );

char *depreciation_prior_depreciation_date(
        char *asset_name,
        char *serial_label,
        char *current_depreciation_date );

double depreciation_amount(
        enum depreciation_method,
        char *service_placement_date,
        depreciation_prior_depreciation_date(),
        char *depreciation_date,
        double cost_basis,
        depreciation_units_produced_current(),
        int estimated_residual_value,
        int estimated_useful_life_years,
        int estimated_useful_life_units,
        int declining_balance_n,
        double prior_accumulated_depreciation );

double depreciation_straight_line(
        char *service_placement_date,
        char *prior_depreciation_date,
        char *depreciation_date,
        double cost_basis,
        double estimated_residual_value,
        int estimated_useful_life_years,
        double prior_accumulated_depreciation );

double depreciation_sum_of_years_digits(
        char *service_placement_date,
        char *prior_depreciation_date,
        char *depreciation_date,
        double cost_basis,
        double estimated_residual_value,
        int estimated_useful_life_years,
        prior_accumulated_depreciation );

double depreciation_double_declining_balance(
        char *service_placement_date,
        char *prior_depreciation_date,
        char *depreciation_date,
        double cost_basis,
        double estimated_residual_value,
        int estimated_useful_life_years,
        double prior_accumulated_depreciation );

double depreciation_n_declining_balance(
        char *service_placement_date,
        char *prior_depreciation_date,
        char *depreciation_date,
        double cost_basis,
        double estimated_residual_value,
        int estimated_useful_life_years,
        int declining_balance_n,
        double prior_accumulated_depreciation );

int depreciation_units_produced_total(
        char *asset_name,
        char *serial_label );

int depreciation_units_produced_current(
        int units_produced_so_far,
        depreciation_units_produced_total() );

double depreciation_units_of_production(
        double cost_basis,
        double estimated_residual_value,
        int estimated_useful_life_units,
        int depreciation_units_produced_current(),
        double prior_accumulated_depreciation );

double depreciation_fraction_of_year(
        char *service_placement_date,
        char *prior_depreciation_date,
        char *depreciation_date_string );

double depreciation_accumulated_depreciation(
        prior_accumulated_depreciation,
        depreciation_amount() );

char *depreciation_subquery_where(
        char *depreciation_date );

TRANSACTION *depreciation_transaction(
        entity_self->entity->full_name,
        entity_self->entity->street_address,
        predictive_transaction_date_time(
            depreciation_date ),
        depreciation_amount(),
        account_depreciation_expense(),
        account_accumulated_depreciation() );

void depreciation_insert(
        FILE *insert_pipe,
        char *asset_name,
        char *serial_label,
        char *depreciation_date,
        int depreciation_units_produced_current(),
        double depreciation_amount,
        char *depreciation_transaction()->
            transaction_date_time );

void depreciation_list_negate_depreciation_amount(
        LIST *depreciation_list );

LIST *depreciation_transaction_list(
        fixed_asset_purchase_depreciation_list() );
```

straight_line,
double_declining_balance,
n_declining_balance,
sum_of_years_digits,
units_of_production,
not_depreciated

PREPAID_ASSET_PURCHASE

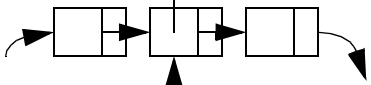



INVENTORY
char *inventory_name
double retail_price
int reorder_quantity
char *inventory_account_name
char *cost_of_goods_sold_account_name
LIST *inventory_purchase_list
INVENTORY_BALANCE *last_inventory_balance
LIST *inventory_balance_list
LIST *inventory_sale_list
INVENTORY_BALANCE
int quantity_on_hand;
double average_unit_cost;
double total_cost_balance;
INVENTORY_PURCHASE *inventory_purchase;
INVENTORY_SALE *inventory_sale;
INVENTORY_SALE
INVENTORY_PURCHASE

PredictiveBooks Data Structure

INVENTORY_PURCHASE

char *	full_name;	int	layer_consumed_quantity;
char *	street_address;	double	average_unit_cost;
char *	purchase_date_time;	char *	arrived_date_time;
char *	inventory_name;	char *	inventory_account_name;
int	ordered_quantity;	LIST *	inventory_purchase_return_list;
double	unit_cost;		
double	capitalized_unit_cost;		
double	extended_cost;		
int	arrived_quantity;		
int	missing_quantity;		
int	quantity_on_hand;		



INVENTORY_PURCHASE_RETURN

char *	return_date_time
int	returned_quantity
double	sales_tax
char *	transaction_date_time
TRANSACTION *	transaction

TRANSACTION

INVENTORY_SALE

char *	full_name;	double	extended_price;
char *	street_address;	double	cost_of_goods_sold;
char *	sale_date_time;	char *	completed_date_time;
char *	inventory_name;	char *	inventory_account_name;
int	sold_quantity;	char *	cost_of_goods_sold_account_name;
double	retail_price;	LIST *	inventory_sale_return_list;
double	discount_amount;	LIST *	layer_inventory_purchase_list;

INVENTORY_SALE_RETURN

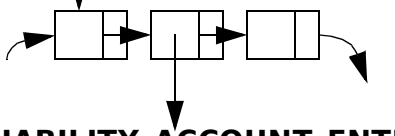
char *	return_date_time;
int	returned_quantity;
char *	transaction_date_time;
TRANSACTION *	transaction;

TRANSACTION

PredictiveBooks Data Structure

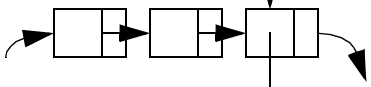
LIABILITY

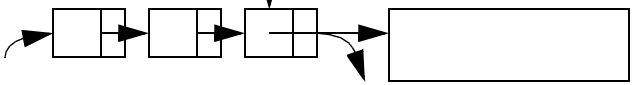
```
/* Input */
int starting_check_number;
LIST *liability_account_entity_list();
LIST *liability_current_account_list();

LIST *input_entity_list =
        entity_full_street_list(
            full_name_list,
            street_address_list );

/* Process */
LIST *liability_tax_redirect_account_list(
        liability_current_account_list(),
        liability_account_entity_list() );

LIST *liability_entity_list(
        liability_tax_redirect_account_list()
            /* liability_account_list */,
        input_entity_list,
        dialog_box_payment_amount );

LIST *liability_after_balance_zero_entity_list(
        liability_entity_list(),
        liability_tax_redirect_account_list()
            /* liability_account_list */ );

LIST *liability_steady_state_entity_list(
        liability_after_balance_zero_entity_list() );

LIST *liability_transaction_list(
        liability_entity_list_steady_state(),
        liability_credit_account_name(
            starting_check_number ),
        account_loss() );
```



LIABILITY_ACCOUNT_ENTITY

```
char *account_name;
ENTITY *entity;
```

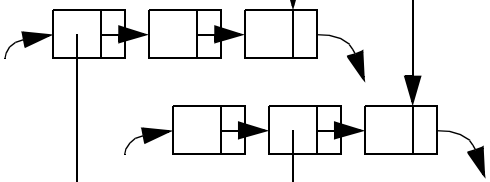




TRANSACTION

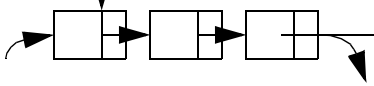
ENTITY

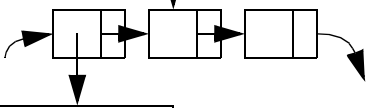
```
char *full_name;
char *street_address;
int check_number;
double dialog_box_payment_amount;

LIST *liability_after_balance_zero_journal_list(
        liability_account_list(),
        full_name,
        street_address );

char *liability_entity_debit_account_name(
        account->account_name );

double liability_entity_amount_due(
        liability_entity_journal_list() );

double liability_entity_payment_amount(
        dialog_box_payment_amount,
        liability_entity_amount_due() );

double liability_entity_loss_amount(
        dialog_box_payment_amount,
        liability_entity_payment_amount() )
{
        return A - B;
}

ENTITY *liability_steady_state_entity(
        entity,
        liability_entity_journal_list() );
```



ACCOUNT

```
char *account_name;

LIST *transaction_after_balance_zero_journal_list(
        account_name );
```



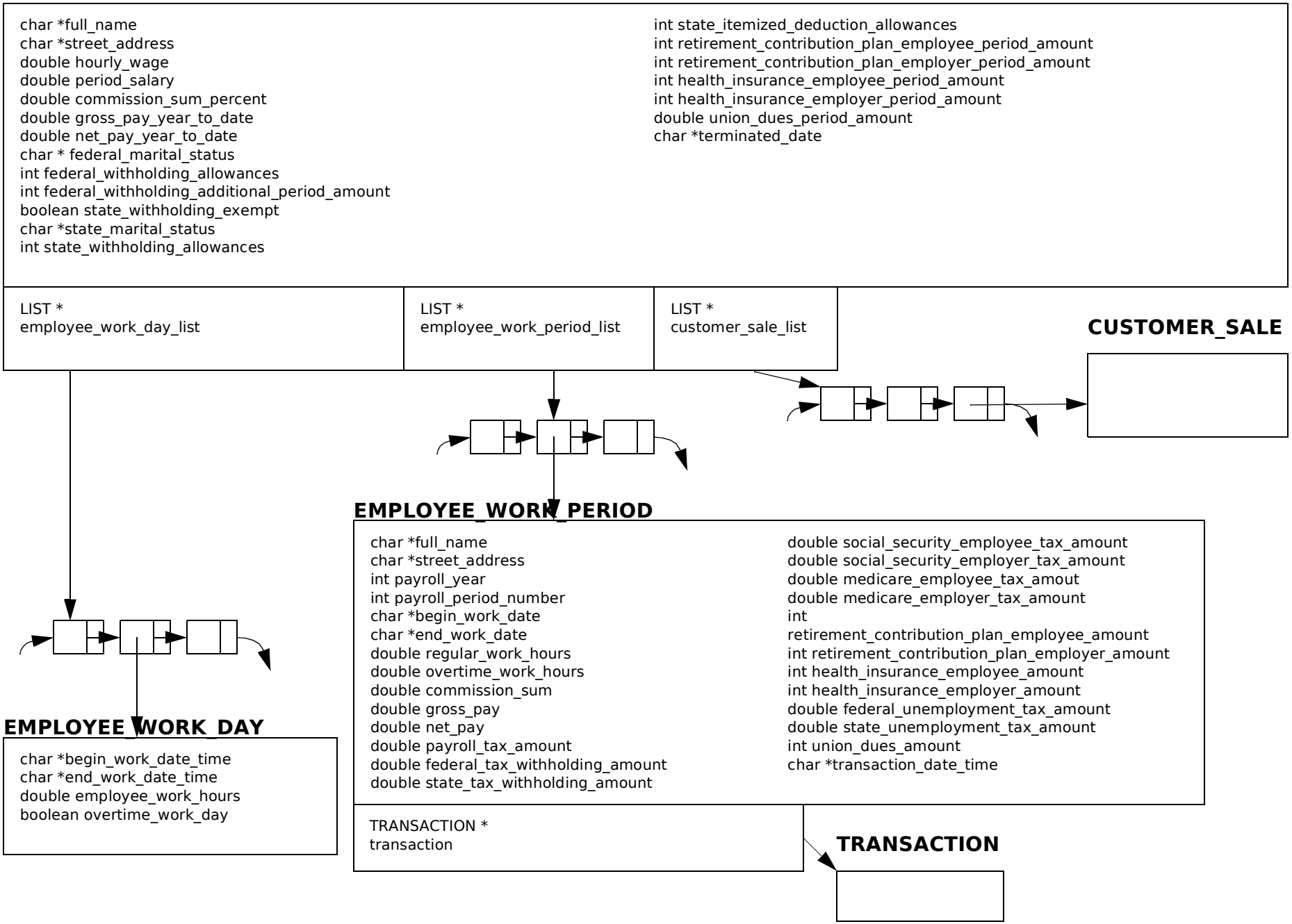


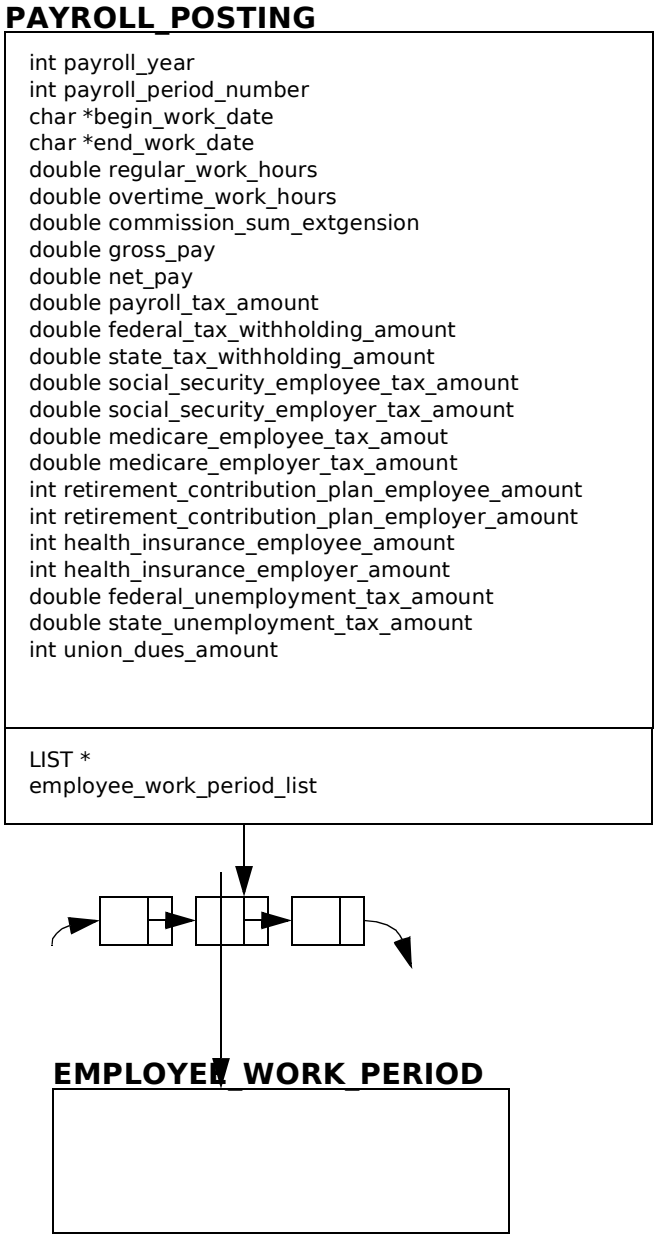
JOURNAL

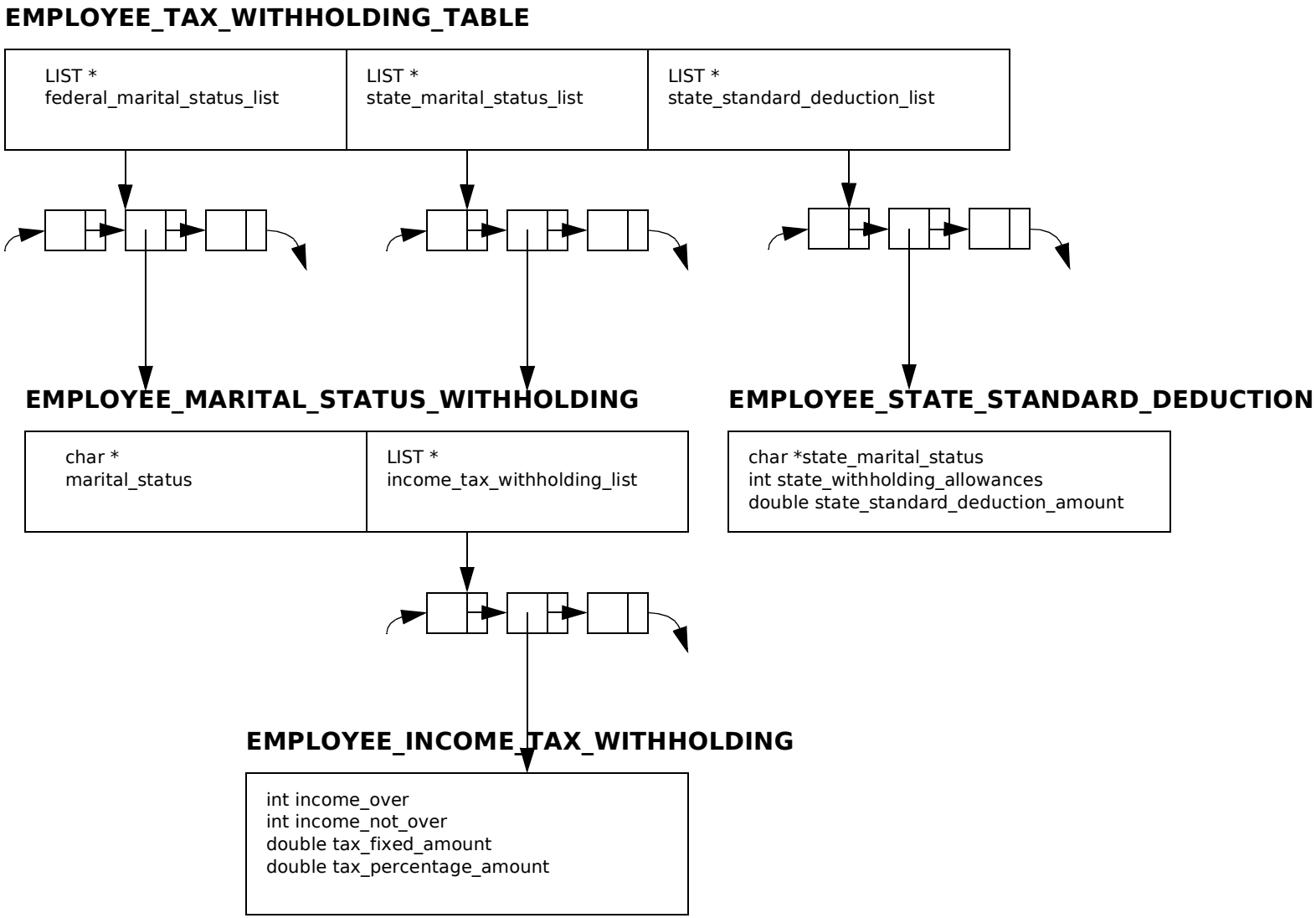
```
char *account_name;
char *full_name;
char *street_address;
double debit_amount;
double credit_amount;
```

PredictiveBooks Data Structure

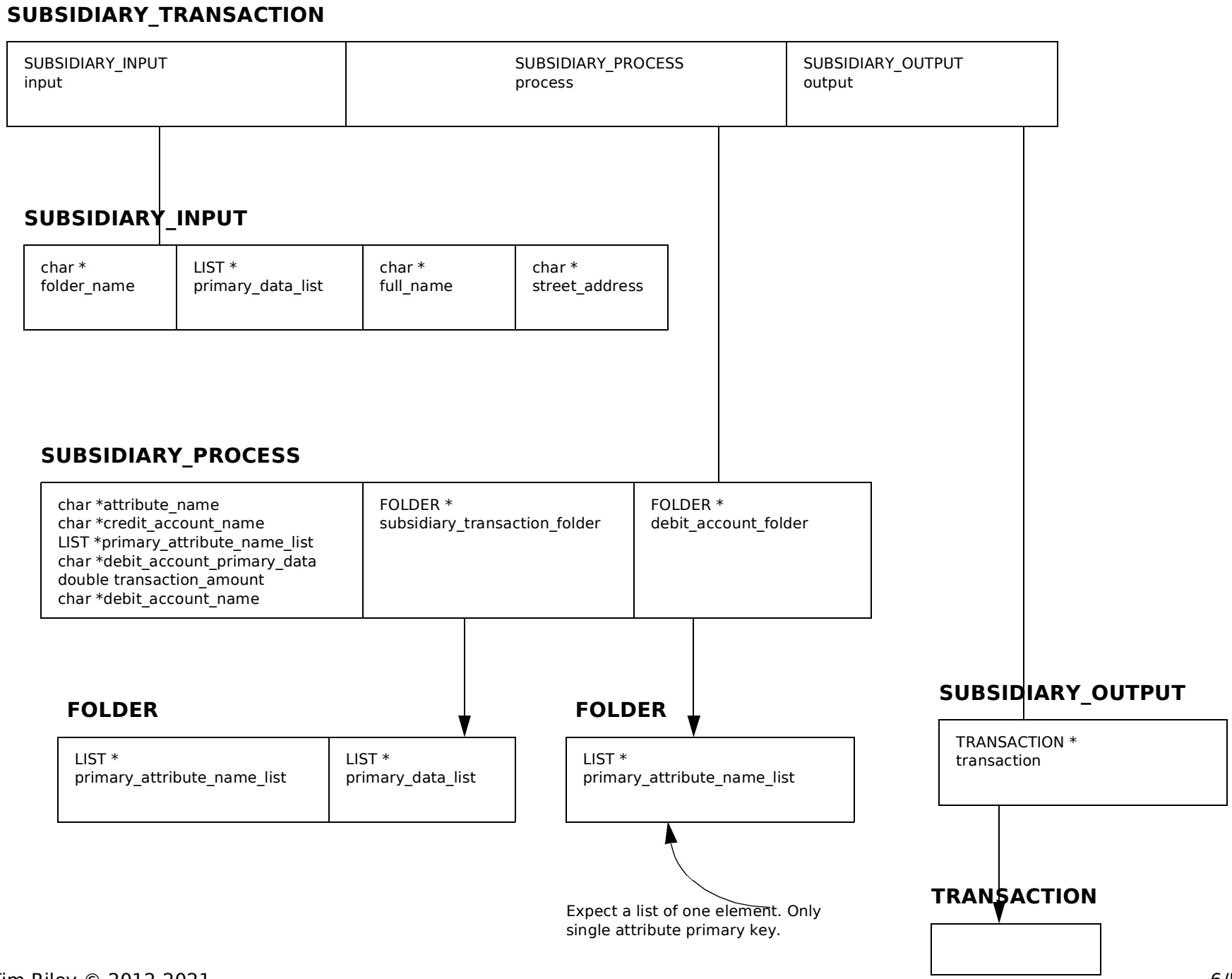
EMPLOYEE

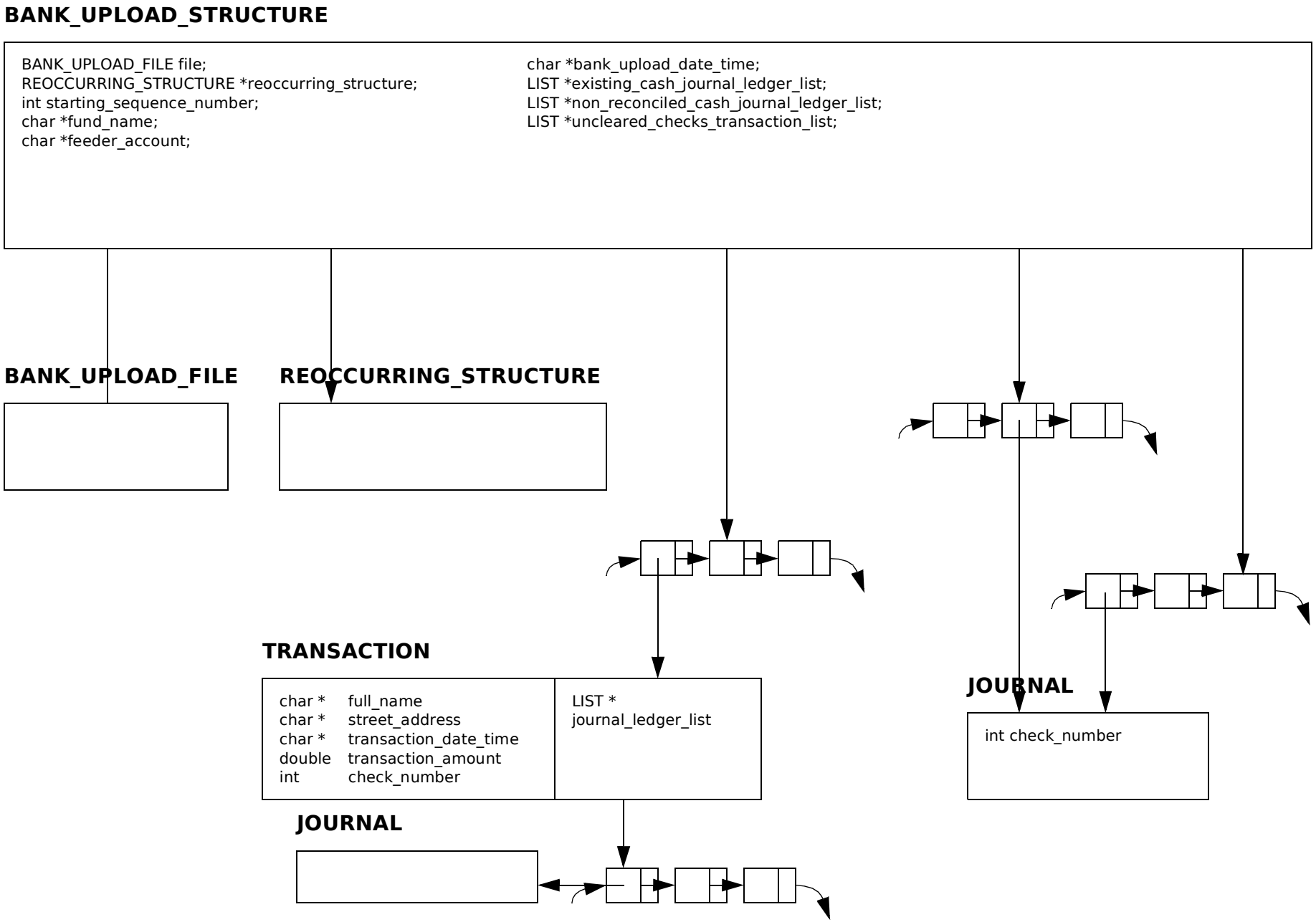


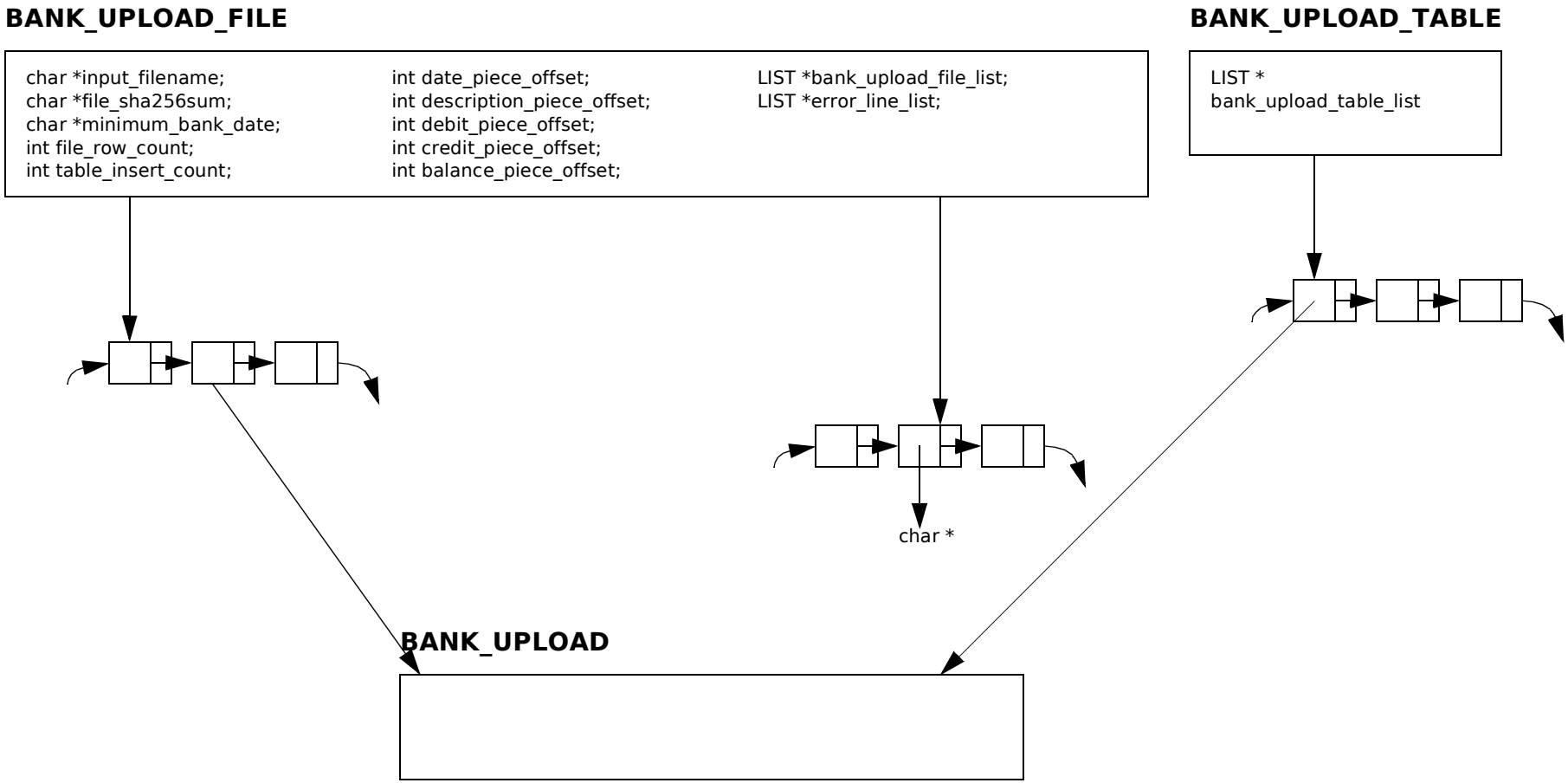

PAYROLL_POSTING
int payroll_year
int payroll_period_number
char *begin_work_date
char *end_work_date
double regular_work_hours
double overtime_work_hours
double commission_sum_extgension
double gross_pay
double net_pay
double payroll_tax_amount
double federal_tax_withholding_amount
double state_tax_withholding_amount
double social_security_employee_tax_amount
double social_security_employer_tax_amount
double medicare_employee_tax_amout
double medicare_employer_tax_amount
int retirement_contribution_plan_employee_amount
int retirement_contribution_plan_employer_amount
int health_insurance_employee_amount
int health_insurance_employer_amount
double federal_unemployment_tax_amount
double state_unemployment_tax_amount
int union_dues_amount
LIST *
employee_work_period_list
EMPLOYEE_WORK_PERIOD


EMPLOYEE_TAX_WITHHOLDING_TABLE
LIST *
federal_marital_status_list
LIST *
state_marital_status_list
LIST *
state_standard_deduction_list
EMPLOYEE_MARITAL_STATUS_WITHHOLDING
char *
marital_status
LIST *
income_tax_withholding_list
EMPLOYEE_STATE_STANDARD_DEDUCTION
char *state_marital_status
int state_withholding_allowances
double state_standard_deduction_amount
EMPLOYEE_INCOME_TAX_WITHHOLDING
int income_over
int income_not_over
double tax_fixed_amount
double tax_percentage_amount

PredictiveBooks Data Structure

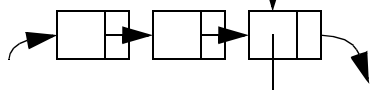



BANK_UPLOAD_STRUCTURE
BANK_UPLOAD_FILE file;
REOCCURRING_STRUCTURE *reoccurring_structure;
int starting_sequence_number;
char *fund_name;
char *feeder_account;
char *bank_upload_date_time;
LIST *existing_cash_journal_ledger_list;
LIST *non_reconciled_cash_journal_ledger_list;
LIST *uncleared_checks_transaction_list;
BANK_UPLOAD_FILE
REOCCURRING_STRUCTURE
TRANSACTION
char * full_name
char * street_address
char * transaction_date_time
double transaction_amount
int check_number
LIST *
journal_ledger_list
JOURNAL
int check_number
JOURNAL


BANK_UPLOAD_FILE
char *input_filename;
char *file_sha256sum;
char *minimum_bank_date;
int file_row_count;
int table_insert_count;
int date_piece_offset;
int description_piece_offset;
int debit_piece_offset;
int credit_piece_offset;
int balance_piece_offset;
LIST *bank_upload_file_list;
LIST *error_line_list;
BANK_UPLOAD_TABLE
LIST *
bank_upload_table_list
char *
BANK_UPLOAD

BANK_UPLOAD

char *bank_date char *bank_description int sequence_number double bank_amount double bank_running_balance	char *fund_name int check_number boolean existing_bank_upload boolean existing_transaction	LIST * reconciled_transaction_list	JOURNAL_LEDGER * cleared_journal_ledger

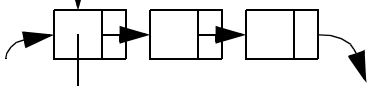


TRANSACTION

JOURNAL_LEDGER

BANK_UPLOAD

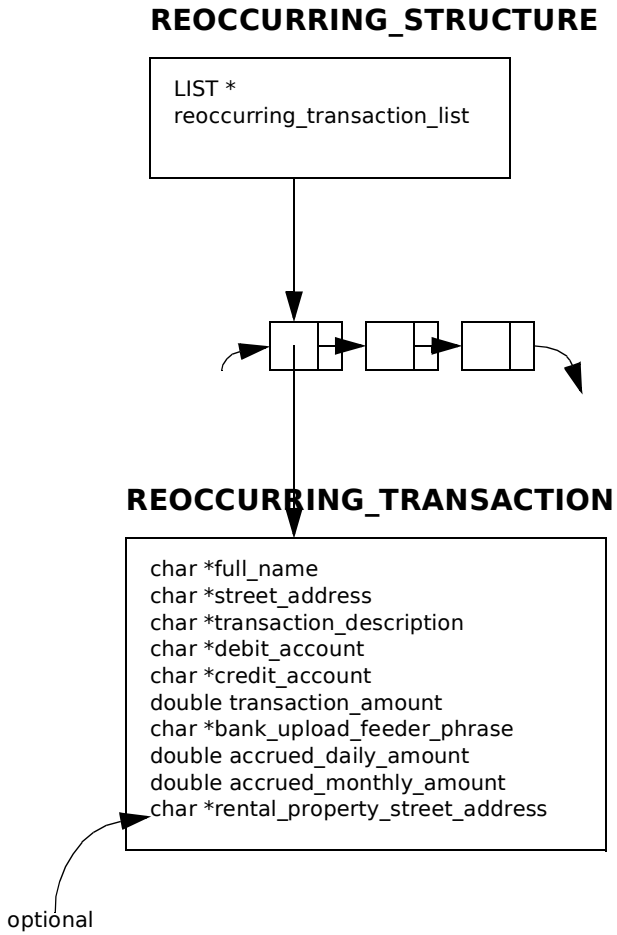
JOURNAL * feeder_check_number_existing_journal(LIST *existing_cash_journal_list, int check_number)	LIST * feeder_match_sum_existing_journal_list(LIST *existing_cash_journal_list, double abs_bank_amount, boolean check_debit)	TRANSACTION * feeder_phrase_match_build_transaction(LIST *reoccurring_transaction_list, char *bank_date, char *bank_description_embedded, double abs_bank_amount)



JOURNAL

JOURNAL

TRANSACTION


REOCCURRING_STRUCTURE
LIST *
reoccurring_transaction_list
REOCCURRING_TRANSACTION
char *full_name
char *street_address
char *transaction_description
char *debit_account
char *credit_account
double transaction_amount
char *bank_upload_feeder_phrase
double accrued_daily_amount
double accrued_monthly_amount
char *rental_property_street_address
optional

PAYPAL

```
/* Usage */
PAYPAL *paypal_fetch(
        spreadsheet_filename,
        date_label );

/* Process */
SPREADSHEET *spreadsheet =
        spreadsheet_fetch(
            spreadsheet_filename,
            date_label );
```

SPREADSHEET

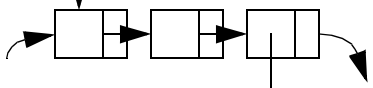
PAYPAL_DATASET

```
/* Input */
char *date_A;
char *time_B;
char *timezone_C;
char *full_name_D;
char *transaction_type_E;
char *transaction_status_F;
char *currency_G;
char *gross_revenue_H;
char *transaction_fee_I;
char *net_revenue_J;
char *from_email_address_K;
char *to_email_address_L;
char *transaction_ID_M;
char *shipping_address_N;
char *address_status_O;
char *item_title_P;
char *item_ID_Q;
char *shipping_handling_amount_R;
char *insurance_amount_S;
char *sales_tax_T;
char *option_1_name_U;
char *option_1_value_V;
char *option_2_name_W;
char *option_2_value_X;
char *reference_txn_ID_Y;
char *invoice_number_Z;
char *custom_number_AA;
char *quantity_AB;
char *receipt_ID_AC;
char *balance_AD;
char *address_line_1_AE;
char *address_2_district_neighborhood_AF;
char *town_city_AG;
char *state_province_AH;
char *zip_code_AI;
char *country_AJ;
char *contact_phone_number_AK;
char *subject_AL;
char *note_AM;
char *country_code_AN;
char *balance_impact_AO;
```

PredictiveBooks Data Structure

SPREADSHEET

```
/* Input */
char *spreadsheet_filename;
char *date_label;

/* Process */
LIST *spreadsheet_column_list(
        spreadsheet_filename,
        date_label );

char *maximum_date;

char *spreadsheet_minimum_date(
        &maximum_date,
        spreadsheet_filename );

char *spreadsheet_heading_data(
        spreadsheet_column_list(),
        input_row,
        heading );
```



SPREADSHEET_COLUMN

```
/* Usage */
SPREADSHEET_COLUMN *spreadsheet_column_new(
        heading,
        piece_offset );

/* Process */
SPREADSHEET_COLUMN *spreadsheet_column_heading_seek(
        spreadsheet_column_list,
        heading );
```

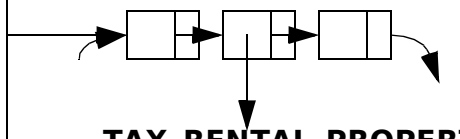
SPREADSHEET_UPLOAD_EVENT

```
/* Input */
char *spreadsheet_filename;

/* Process */
char *spreadsheet_upload_date_time =
        date_time19_now();

char *spreadsheet_upload_file_sha256sum(
        spreadsheet_filename );
```

TAX

```
/* Usage */
TAX *tax_new(
        char *tax_form_string,
        int tax_year );

/* Input */
char *tax_form_string;
int tax_year;

/* Attributes */
```

```
TAX_FORM *tax_form =
        tax_form_fetch();

RECOVERY *recovery;

LIST *tax_rental_property_list =
        tax_rental_property_list_fetch(
            tax_form_string,
            tax_year );
```



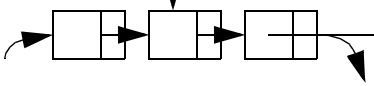
TAX_RENTAL_PROPERTY

TAX_FORM

```
/* Usage */
TAX_FORM *tax_form_fetch(
        char *tax_form_string,
        int tax_year,
        boolean fetch_line_list,
        boolean fetch_account_list,
        boolean fetch_journal_list );

/* Attributes */
char *tax_form_string;

/* Input */
int tax_year;

/* Process */
LIST *tax_form_line_list(
        tax_form_string,
        tax_year,
        fetch_account_list,
        fetch_journal_list );
```

RECOVERY

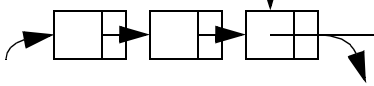
```
double fixed_asset_recovery_amount;
double prior_fixed_asset_recovery_amount;
double property_recovery_amount;
double prior_property_recovery_amount;
double total_recovery_amount;
```



TAX_FORM_LINE

```
/* Usage */
LIST *tax_form_line_list(
        char *tax_form_string,
        int tax_year,
        boolean fetch_account_list,
        boolean fetch_journal_list );

/* Attributes */
char *tax_form_string;
char *tax_form_line_string;
char *tax_form_description;
boolean itemize_accounts;

/* Input */
int tax_year;
```

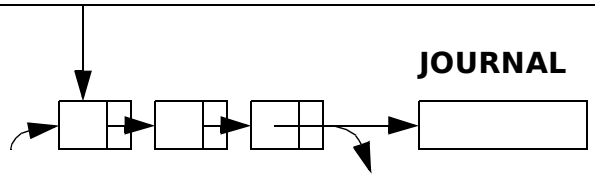
```
/* Process */
LIST *tax_form_line_account_list(
        tax_form_string,
        tax_form_line_string,
        tax_year,
        fetch_journal_list );

/* Operations */
void tax_form_line_list_steady_state(
        tax_form_line_list );

double tax_form_line_total(
        tax_form_line_account_list() );
```



TAX_FORM_LINE_ACCOUNT

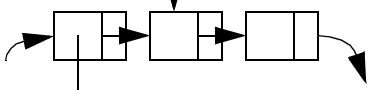
TAX_FORM_LINE_ACCOUNT

```
/* Usage */
LIST *tax_form_line_account_list(
        char *tax_form_string,
        char *tax_form_line,
        int tax_year,
        boolean fetch_journal_list );

/* Attributes */
char *tax_form_string;
char *tax_form_line;
char *account_name;

/* Process */
ACCOUNT *account =
        /* Sets account->accumulate_debit */
        account_fetch(
            account_name );

boolean current_liability =
        subclassification_current_liability(
            account->subclassification_name );

LIST *journal_list =
        journal_year_list(
            tax_year,
            account_name,
            1 /* fetch_transaction_memo */ );

/* Operations */
double tax_form_line_account_total(
        journal_list,
        account->accumulate_debit,
        current_liability );
```



TAX_RENTAL_PROPERTY

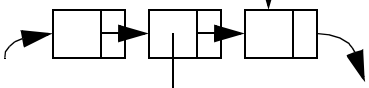
```
/* Usage */
LIST *tax_rental_property_list_fetch(
        char *tax_form_string,
        int tax_year );

/* Attributes */
char  *property_street_address;

/* Process */
LIST *tax_rental_property_string_list(
        tax_rental_property_list );

LIST *tax_form_line_rental_list(
        tax_form_line_list(),
        tax_rental_property_string_list );

double recovery_amount;
double revenue_total [23a];
double mortgage_interest_total [23c];
double depreciation_total [23d];
double expense_total [23e];
double positive_net_income [24];
double loss_total [25];
double tax_rental_property_total();
```



TAX_FORM_LINE_RENTAL

```
char *tax_form_line;
char *tax_form_description;

enum tax_form_line_category;
LIST *tax_form_line_account_list();

LIST *rental_property_list;
double tax_form_line_rental_property_total();
```

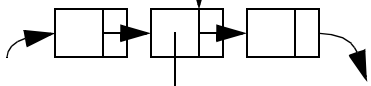
revenue
mortgage_interest_paid
depreciation_expense
operating_expense
loss



TAX_FORM_LINE_ACCOUNT

INVESTMENT_ACCOUNT

```
ENTITY *financial_entity =
        entity_new(
            full_name,
            street_address );

char *account_number;
char *investment_classification;
char *investment_purpose;

LIST *investment_account_balance_list(
        full_name,
        street_address,
        account_number );
```



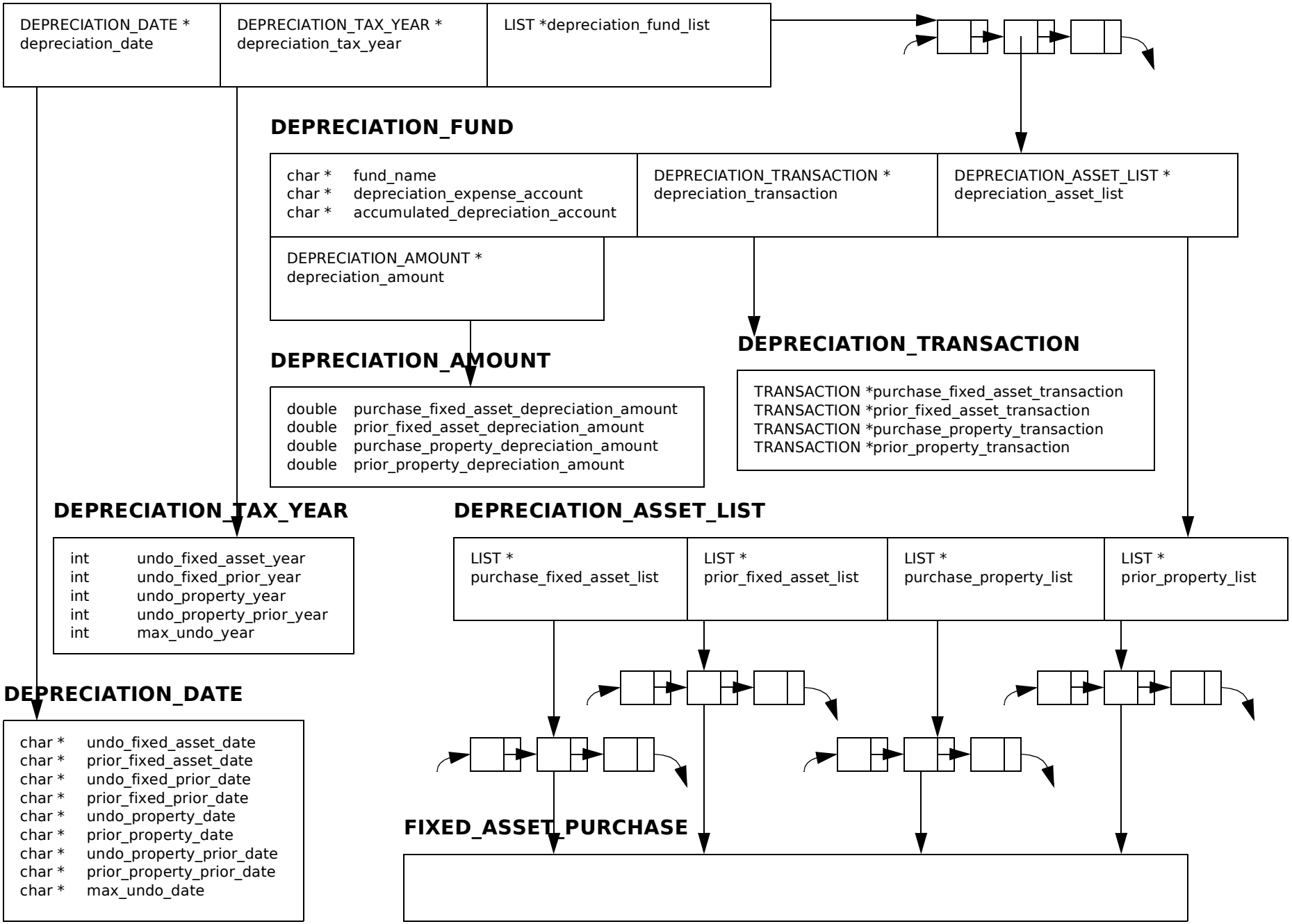
ACCOUNT_BALANCE

```
char *date
double balance;
double balance_change;
int balance_change_percent;
```

TRANSACTION

ACCOUNT_BALANCE_EQUITY

```
char *date_time
char *investment_operation
double share_price
doubel share_quantity_change
double share_quantity_balance
double cash_in
double market_value
double book_value_change
double book_value_balance
double moving_share_price
double unrealized_gain_balance
double unrealized_gain_change
double reaqlized_gain
boolean table_first_row
char *transaction_date_time
TRANSACTION *
transaction
```

PredictiveBooks Data Structure

DEPRECIATION_STRUCTURE